

Contents

1 Kabya	2
1.1 binary_lifting	2
1.2 centroid	2
1.3 convex_hull	2
1.4 crt	2
1.5 dsu	3
1.6 extendedEuclidean	3
1.7 fenwick_tree	3
1.8 lcs	3
1.9 millerRabin	3
1.10 pbds	3
1.11 segmentTree	3
1.12 segmentTreeLazy	4
1.13 zzsublime_build	4
2 Shihab	4
2.1 basic	4
2.2 modu	5
2.3 prime	5
3 Tamzid	6
3.1 StressTest	6
3.1.1 checker	6
3.1.2 gen	6
3.1.3 script	6
3.2 KMP	6
3.3 area_intersectingpoint	7
3.4 bellmanford	7
3.5 convexhull	7
3.6 moalgo	8
3.7 ncr	8
3.8 polyhash	8
3.9 scckosaraju	9
3.10 sparsetable	9
3.11 tree	9
3.12 treediameter	9
4 zFrom Sifat Bhai	10
4.1 Data Structure	10
4.1.1 BIT_range_update	10
4.1.2 CHT	10
4.1.3 dynamic_cht	10
4.1.4 gp_hash_table	10
4.1.5 persistent_segtree	10
4.2 Geometry	11
4.2.1 2D Primitive	11
4.2.1.1 Angle	11
4.2.1.2 On Segment	11
4.2.1.3 Point Sort	11
4.2.1.4 Point	11
4.2.1.5 Side Of	12
4.2.2 3D	12
4.2.2.1 3D Convex Hull	12
4.2.2.2 Point3D	12
4.2.2.3 Polyhedron Volume	12

4.2.2.4 Spherical Distance	13
4.2.3 Circle	13
4.2.3.1 Circle Intersection	13
4.2.3.2 Circle Polygon Intersection	13
4.2.3.3 Circle Tangents	13
4.2.4 Polygon	13
4.2.4.1 Hull Diameter	13
4.2.4.2 Line Hull Intersection	13
4.2.4.3 Polygon Center	14
4.2.5 Minimum Enclosing Circle	14
4.2.6 Point In Polygon	14
4.2.7 sweep	14
4.3 Graph	15
4.3.1 2Sat	15
4.3.2 Centroid_decomp	16
4.3.3 articulation_point	16
4.3.4 bcc	16
4.3.5 bridge_tree	16
4.3.6 dsu_on_tree	17
4.3.7 euler_path	17
4.3.8 hld	17
4.3.9 kuhn	18
4.3.10 mcmf	18
4.4 Math	18
4.4.1 FloorSum	18
4.4.2 NOD	19
4.4.3 Pollard Rho	19
4.4.4 catalan	19
4.4.5 crt	19
4.4.6 derangement	19
4.4.7 diophantine	19
4.4.8 discrete_log	19
4.4.9 factorial_mod_p	20
4.4.10 fft	20
4.4.11 gauss_elimination	20
4.4.12 gen_all_k_combs	20
4.4.13 matrix_expo	21
4.4.14 next_lexicographical_k_comb	21
4.4.15 ntt	21
4.4.16 seg_sieve	21
4.4.17 stirling	21
4.4.18 stirling_number_of_the_second_kind	21
4.4.19 totient	21
4.4.20 xor_basis	22
4.5 String	22
4.5.1 aho_corasick	22
4.5.2 hash	22
4.5.3 hash_segtree	22
4.5.4 manacher	23
4.5.5 palindromic_tree	23
4.5.6 persistent_trie	23
4.5.7 suffix_array	24
4.5.8 suffix_automata	24
4.5.9 trie	25
4.5.10 z_algo	25

1 Kabya**1.1 binary_lifting****class** TreeAncestor

```
{
public:
    vector<vector<int>> dp;

    TreeAncestor(vector<int> &parent)
    {
        int n = parent.size();
        dp.resize(17, vector<int>(n, -1));

        for (int i = 0; i < n; i++)
            dp[0][i] = parent[i];

        for (int i = 1; i < 17; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (dp[i - 1][j] != -1)
                    dp[i][j] = dp[i - 1][dp[i - 1][j]];
            }
        }

        int getKthAncestor(int node, int k)
        {
            for (int p = 0; p < 17; p++)
            {
                if (k & (1LL << p))
                {
                    node = dp[p][node];
                    if (node == -1)
                        break;
                }
            }
            return node;
        }
    };
};
```

1.2 centroid

```
const int M = 1e9 + 7, N = 100005;
vector<int> g[N], subTree(N), deleted(N);
vector<char> color(N);
// char s[] = {'A', 'B'};
int getSubtreeSize(int node = 1, int p = -1)
{
    subTree[node] = 1;
    for (auto &it : g[node])
    {
        if (it == p || deleted[it])
            continue;
        subTree[node] += getSubtreeSize(it, node);
    }
    return subTree[node];
}

int getCentroid(int node, int treeSize, int p = -1)
{
    for (auto &it : g[node])
    {
        if (it == p || deleted[it])
            continue;
        if (subTree[it] * 2 > treeSize)
            return getCentroid(it, treeSize, node);
    }
    return node;
}

void magic(int node = 1, char c = 'A')
```

```
{
    int centroid = getCentroid(node,
        getSubtreeSize(node));
    deleted[centroid] = 1;
    color[centroid] = c;
    for (auto &it : g[centroid])
    {
        if (!deleted[it])
        {
            magic(it, c + 1);
        }
    }
}

void kire()
{
    int n;
    cin >> n;
    for (int i = 0; i < n - 1; i++)
    {
        int u, v;
        cin >> u >> v;
        gInput;
    }
    magic();
    for (int i = 0; i < n; i++)
    {
        cout << color[i + 1] << gap;
    }
}
```

1.3 convex_hull

```
#include<bits/stdc++.h>
using namespace std;
struct Point {
    int x, y;
    bool operator < (Point P) const {
        if (x == P.x) return y < P.y;
        else return x < P.x;
    }
    bool operator == (Point P) const {
        return (x == P.x && y == P.y);
    }
};

bool cw(Point a, Point b, Point c) {
    return a.x * (c.y - b.y) + b.x * (a.y - c.y) + c.x *
        (b.y - a.y) > 0;
}

bool ccw(Point a, Point b, Point c) {
    return a.x * (c.y - b.y) + b.x * (a.y - c.y) + c.x *
        (b.y - a.y) < 0;
}

bool cll(Point a, Point b, Point c) {
    return a.x * (c.y - b.y) + b.x * (a.y - c.y) + c.x *
        (b.y - a.y) == 0;
}

vector<Point> convex_hull(vector<Point> points) {
    if (points.size() <= 2) return points;
    sort(points.begin(), points.end());
    vector<Point> up, down;
    Point first = points[0], last = points.back();
    up.push_back(first); down.push_back(first);
```

```
for (int i = 1; i < points.size(); i++) {
    if (i == points.size() - 1 || !ccw(first, points[i],
        last)) {
        up.pop_back();
        while (up.size() >= 2 && ccw(up[up.size() - 2],
            up[up.size() - 1], points[i])) {
            up.pop_back();
        }
        up.push_back(points[i]);
    }
    if (i == points.size() - 1 || !cw(first, points[i],
        last)) {
        down.pop_back();
        while (down.size() >= 2 && cw(down[down.size() - 2],
            down[down.size() - 1], points[i])) {
            down.pop_back();
        }
        down.push_back(points[i]);
    }
}

points.clear();
for (int i = 0; i < up.size(); i++) {
    points.push_back(up[i]);
}
for (int i = 0; i < down.size(); i++) {
    points.push_back(down[i]);
}

sort(points.begin(), points.end());
points.resize(unique(points.begin(), points.end()) -
    points.begin());
return points;
}

int main()
{
    freopen("input.txt", "r", stdin);
    freopen("output.txt", "w", stdout);

    int n;
    cin >> n;
    vector<Point> points(n);
    for (int i = 0; i < n; i++) {
        cin >> points[i].x >> points[i].y;
    }
    vector<Point> convex_hull_vector = convex_hull(points);
    for (auto x : convex_hull_vector) {
        cout << x.x << " " << x.y << '\n';
    }
    return 0;
}
```

1.4 crt

```
#include <iostream>
#include <vector>
using namespace std;
// Extended Euclidean Algorithm to find modular inverse
```

```

int modInverse(int a, int m) {
    int m0 = m, x0 = 0, x1 = 1;
    if (m == 1) return 0;
    while (a > 1) {
        int q = a / m;
        int t = m;
        m = a % m; a = t;
        t = x0;
        x0 = x1 - q * x0;
        x1 = t;
    }
    if (x1 < 0) x1 += m0;
    return x1;
}

// Chinese Remainder Theorem
int chineseRemainder(const vector<int>& num, const
    vector<int>& rem) {
    int prod = 1;
    for (int n : num) prod *= n;

    int result = 0;
    for (int i = 0; i < num.size(); i++) {
        int pp = prod / num[i];
        result += rem[i] * modInverse(pp, num[i]) * pp;
    }
    return result % prod;
}

int main() {
    vector<int> num = {3, 4, 5}; // moduli
    vector<int> rem = {2, 3, 1}; // remainders

    cout << "x = " << chineseRemainder(num, rem) << endl;
    return 0;
}

```

1.5 dsu

```

int find(int n)
{
    if (par[n] == n)
        return n;
    return par[n] = find(par[n]);
}

void unnoyon(int a, int b)
{
    a = find(a);
    b = find(b);
    if (a != b)
        par[a] = b;
}

```

1.6 extendedEuclidean

```

typedef pair<int, int> pii;
#define x first
#define y second

pii extendedEuclid(int a, int b) { // returns x, y | ax +
    by = gcd(a,b)
    if (b == 0) return pii(1, 0);
    else {
        pii d = extendedEuclid(b, a % b);
        return pii(d.y, d.x - d.y * (a / b));
    }
}

```

1.7 fenwick_tree

```

#include <bits/stdc++.h>
using namespace std;
int bit[200005];
void update(int i, int delta)
{
    i++; // hack to allow 0-based indexing in fenwick
    while (i <= n) {
        bit[i] += delta;
        i += i & -i;
    }
}

int query(int l, int r)
{
    r++;
    int sum = 0;
    while (r > 0) {
        sum += bit[r];
        r -= r & -r;
    }
    l++;
    while (l > 0) {
        sum -= bit[l];
        l -= l & -l;
    }
    return sum;
}

```

1.8 lcs

```

int lcs(string x, string y, int n, int m)
{
    int dp[n + 1][m + 1] = {};
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= m; j++)
        {
            if (x[i - 1] == y[j - 1])
                dp[i][j] = 1 + dp[i - 1][j - 1];
            else
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    }
    return dp[n][m];
}

```

1.9 millerRaben

```

typedef long long ll;
ll mulmod(ll a, ll b, ll c) {
    ll x = 0, y = a % c;
    while (b) {
        if (b & 1) x = (x + y) % c;
        y = (y << 1) % c;
        b >>= 1;
    }
    return x % c;
}

ll fastPow(ll x, ll n, ll MOD) {
    ll ret = 1;
    while (n) {
        if (n & 1) ret = mulmod(ret, x, MOD);
        x = mulmod(x, x, MOD);
        n >>= 1;
    }
    return ret;
}

bool isPrime(ll n) {
    ll d = n - 1;

```

```

int s = 0;
while (d % 2 == 0) {
    s++;
    d >>= 1;
}
// It's guranteed that these values will work for any
// number smaller than 3*10**18 (3 and 18 zeros)
int a[9] = { 2, 3, 5, 7, 11, 13, 17, 19, 23 };
for (int i = 0; i < 9; i++) {
    bool comp = fastPow(a[i], d, n) != 1;
    if (comp) for (int j = 0; j < s; j++) {
        ll fp = fastPow(a[i], (1LL << (1LL)) * d, n);
        if (fp == n - 1) {
            comp = false;
            break;
        }
    }
    if (comp) return false;
}
return true;
}

```

1.10 pbds

```

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
// Template for ordered set (acts like set but with
// additional functions)
template <class T>
using ordered_set = tree<T, null_type, less<T>,
    rb_tree_tag, tree_order_statistics_node_update>;
// For ordered multiset (allows duplicates)
template <class T>
using ordered_multiset = tree<T, null_type,
    less_equal<T>, rb_tree_tag,
    tree_order_statistics_node_update>;
// For ordered set of pairs
ordered_set<pair<int, int>> osp;
// For ordered set with custom comparator
struct custom_compare {
    bool operator() (const pair<int, int>& a, const
        pair<int, int>& b) const {
        if (a.first != b.first) return a.first < b.first;
        return a.second < b.second;
    }
};
tree<pair<int, int>, null_type, custom_compare,
    rb_tree_tag, tree_order_statistics_node_update>
    custom_ordered_set;

// KEY FUNCTIONS
osp.insert(5); // Insert element
osp.erase(5); // Erase element
osp.find(2)
*osp.find by order(0)
osp.order_of_key(2)

s.erase(s.lower_bound(val), s.upper_bound(val));

gp_hash_table<int, int> fast_map;
gp_hash_table<int, null_type> fast_set;

```

1.11 segmentTree

```

#include <bits/stdc++.h>
using namespace std;
#define int long long

```

```

#define tata return 0
#define all(v) v.begin(), v.end()
#define endl '\n'
#define gap ' '

const int M = 1e9 + 7;

struct segTree
{
    int n;
    vector<int> v, a;

    segTree(int _n, int _a[])
    {
        n = _n;
        a.assign(_a, _a + n);
        v.resize(4 * n);
        build(0, n - 1, 1);
    }

    void build(int tl, int tr, int node)
    {
        if (tl == tr)
            v[node] = a[tl];
        else
        {
            int mid = (tl + tr) / 2;
            build(tl, mid, 2 * node);
            build(mid + 1, tr, 2 * node + 1);
            v[node] = v[node * 2] + v[node * 2 + 1];
        }
    }

    int sum(int tl, int tr, int l, int r, int node)
    {
        if (l > r)
            return 0;
        if (l == tl && r == tr)
            return v[node];
        int mid = (tl + tr) / 2;
        return sum(tl, mid, l, min(r, mid), node * 2) +
            sum(mid + 1, tr, max(l, mid + 1), r, node * 2 + 1);
    }

    void update(int tl, int tr, int pos, int val, int node)
    {
        if (tl == tr)
            v[node] = val;
        else
        {
            int mid = (tl + tr) / 2;
            if (pos <= mid)
                update(tl, mid, pos, val, node * 2);
            else
                update(mid + 1, tr, pos, val, node * 2 + 1);
            v[node] = v[node * 2] + v[node * 2 + 1];
        }
    }
};

int32_t main()
{
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    cout.tie(NULL);

    int n;
    cin >> n;
    int a[n];
    for (auto &it : a)
        cin >> it;
    segTree tree(n, a);

```

```

cout << tree.sum(0, n - 1, 0, 3, 1) << endl;
tree.update(0, n - 1, 2, 12, 1);
cout << tree.sum(0, n - 1, 2, 2, 1) << endl;
}

```

1.12 segmentTreeLazy

```

#include <bits/stdc++.h>
// #include <ext/pb_ds/assoc_container.hpp>
// #include <ext/pb_ds/tree_policy.hpp>
using namespace std;
// using namespace __gnu_pbds;
// #define ordered_set tree<int, null_type, less<int>,
// rb_tree_tag, tree_order_statistics_node_update>
//mt19937 rng(chrono::high_resolution_clock::now().time_
// since epoch().count());
#define gInput g[u].push_back(v), g[v].push_back(u);
#define int long long
#define pii pair<int, int>
#define tata return 0
#define all(v) v.begin(), v.end()
#define endl '\n'
#define gap ' '

const int M = 1e9 + 7;
const int N = 2e5 + 7;
struct segTreeL
{
    int n;
    vector<int> v, a;
    segTreeL(int _n, int _a[])
    {
        n = _n;
        v.resize(4 * n);
        a.assign(_a, _a + n);
        build(0, n - 1, 1);
    }

    void build(int tl, int tr, int node)
    {
        if (tl == tr)
            v[node] = a[tl];
        else
        {
            int mid = (tl + tr) / 2;
            build(tl, mid, 2 * node);
            build(mid + 1, tr, 2 * node + 1);
            v[node] = 0;
        }
    }

    int query(int tl, int tr, int pos, int node)
    {
        if (tl == tr)
            return v[node];
        else
        {
            int mid = (tl + tr) / 2;
            if (pos <= mid)
                return v[node] + query(tl, mid, pos, 2 * node);
            else
                return v[node] + query(mid + 1, tr, pos, 2 * node + 1);
        }
    }

    void update(int tl, int tr, int l, int r, int val, int node)
    {
        if (l > r)
            return;
        if (l == tl && r == tr)

```

```

{
    v[node] += val;
    return;
}

int mid = (tl + tr) / 2;

update(tl, mid, l, min(r, mid), val, 2 * node);
update(mid + 1, tr, max(l, mid + 1), r, val, 2 * node + 1);
}

};

void kire()
{
    int n;
    cin >> n;
    int a[n];
    for (auto &it : a)
        cin >> it;

    segTreeL tree(n, a);

    tree.update(0, n - 1, 2, n - 1, 3, 1);
    cout << tree.query(0, n - 1, 3, 1) << endl;
    cout << tree.query(0, n - 1, 4, 1) << endl;
}

int32_t main()
{
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    cout.tie(NULL);
    // freopen("input.txt", "r", stdin);
    // freopen("output.txt", "w", stdout);
    int t = 1;
    // cin>>t;

    for (int i = 1; i <= t; i++)
    {
        // cout<<"Case "<<i<<": ";
        kire();
    }

    tata;
}

```

1.13 zzsublime_build

```

{
    "cmd" : ["g++ -std=c++14 $file_name -o $file_base_name &&
        timeout 4s ./file_base_name<input.txt>output.txt"],
    "selector" : "source.c",
    "shell" : true,
    "working_dir" : "$file_path"
}

```

2 Shihab

2.1 basic

```

/*
AUTHOR: shihab
Time Created: 17:14:32
Date Created: 07 May 2025
*/

#include <bits/stdc++.h>
using namespace std;
#define ll long long

```

```
#define nl "\n"
#define pb(x) push_back(x)
#define vll vector<ll>
#define vlll vector<vll>
#define loop(i,j,n) for(int i=j;i<n;i++)
#define getv(v,n) vector<ll> v(n); loop(i,0,n) cin >>
    v[i];
#define vpll vector<pair<ll,ll>>
#define pll pair<ll,ll>
#define clr(a) memset((a),0,sizeof(a))
#define all(x) (x).begin(),(x).end()
#define mkunique(x)
    sort(all(x));(x).erase(unique(all(x)),(x).end())
#define fi first
#define se second
#define rrr return;
#define mp make_pair
#define si(x) int(x.size())
#define yes cout<<"YES"<<endl;
#define no cout<<"NO"<<endl;
bool odd(ll num) {return ((num & 1) == 1);}
bool even(ll num) {return ((num & 1) == 0);}
const int mod=1e9+7, INF=15<<26;

#ifdef LOCAL

void __print(int x) { cerr << x; }
void __print(long x) { cerr << x; }
void __print(long long x) { cerr << x; }
void __print(unsigned x) { cerr << x; }
void __print(unsigned long x) { cerr << x; }
void __print(unsigned long long x) { cerr << x; }
void __print(float x) { cerr << x; }
void __print(double x) { cerr << x; }
void __print(long double x) { cerr << x; }
void __print(char x) { cerr << '\'' << x << '\''; }
void __print(const char *x) { cerr << "\"" << x << "\""; }
    }
void __print(const string &x) { cerr << "\"" << x <<
    "\""; }
void __print(bool x) { cerr << (x ? "true" : "false"); }

template <typename T, typename V>
void __print(const pair<T, V> &x) { cerr << '{';
    __print(x.first); cerr << ", "; __print(x.second);
    cerr << '}' ; }

template <typename T>
void __print(const T &x) {
    int f = 0;
    cerr << '{';
    for (auto &i : x) cerr << (f++ ? ", " : ""),
        __print(i);
    cerr << '}' ;
}

void __print() { cerr << "]\n"; }
template <typename T, typename... V>
void __print(T t, V... v) { __print(t); if (sizeof...(v))
    cerr << ", "; __print(v...); }

#define dbg(x...) cerr << "[" << #x << "] = [";
    __print(x);
#define dbg(x...) 42
#endif

void sol() {
    ll
```

```
}
signed main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    ll t = 1;
    cin >> t;
    while (t--) {
        sol();
    }
}

2.2 modu

// Basic Operations
ll add(ll a, ll b, ll mod) {
    return (a + b) % mod;
}
ll sub(ll a, ll b, ll mod) {
    return (a - b + mod) % mod;
}
ll mul(ll a, ll b, ll mod) {
    return (a * b) % mod;
}
// Modular Exponentiation (a^b % mod)
ll power(ll a, ll b, ll mod) {
    ll res = 1;
    a %= mod;
    while (b > 0) {
        if (b & 1) res = mul(res, a, mod);
        a = mul(a, a, mod);
        b >>= 1;
    }
    return res;
}
// Extended Euclidean Algorithm
ll egcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    ll x1, y1;
    ll g = egcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}
// Modular Inverse using Extended Euclidean
ll inv(ll a, ll mod) {
    ll x, y;
    ll g = egcd(a, mod, x, y);
    assert(g == 1); // Ensure a and mod are coprime
    return (x % mod + mod) % mod;
}
// Fast Modular Inverse using Fermat's Little Theorem
    (prime mod only)
ll invf(ll a, ll mod) {
    return power(a, mod - 2, mod);
}
// Precomputation for factorials and inverse factorials
vector<ll> fact, infact;
void facto(int max_n, ll mod) {
    fact.resize(max_n + 1);
    infact.resize(max_n + 1);
    fact[0] = 1;
    for (int i = 1; i <= max_n; i++) {
        fact[i] = mul(fact[i - 1], i, mod);
    }
    infact[max_n] = inv(fact[max_n], mod);
```

```
for (int i = max_n - 1; i >= 0; i--) {
    infact[i] = mul(infact[i + 1], i + 1, mod);
}
// Combination (n choose k) using precomputed factorials
ll nck(int n, int k, ll mod) {
    if (k < 0 || k > n) return 0;
    return mul(fact[n], mul(infact[k], infact[n - k],
        mod), mod);
}

2.3 prime

// Maximum limit for the basic sieve; adjust as needed.
const int MAXN = 1000001;
// Global containers for our sieve and list of primes.
vector<bool> isPrimeSieve(MAXN, true);
vector<int> primes;
// Build the sieve of Eratosthenes to mark prime numbers
    up to MAXN.
void sie(){
    isPrimeSieve[0] = isPrimeSieve[1] = false;
    for (int i = 2; i < MAXN; i++) {
        if (isPrimeSieve[i]) {
            primes.push_back(i);
            if ((ll)i * i < MAXN) {
                for (ll j = (ll)i * i; j < MAXN; j += i)
                    isPrimeSieve[j] = false;
            }
        }
    }
}
// Compute the prime factors of n.
// Returns a vector of pairs {prime factor, exponent}.
vector<pair<ll, ll>> primeFactors(ll n) {
    vector<pair<ll, ll>> factors;
    for (auto p : primes) {
        if ((ll)p * p > n) break;
        if (n % p == 0) {
            ll count = 0;
            while (n % p == 0) {
                ++count;
                n /= p;
            }
            factors.push_back({p, count});
        }
    }
    if (n > 1) factors.push_back({n, 1});
    return factors;
}

// numPF(N): Count the number of distinct prime factors
    of N.
ll numPF(ll N) {
    auto factors = primeFactors(N);
    return factors.size(); // each pair represents a
        distinct prime factor.
}

// numDiv(N): Count the number of divisors of N.
// If N = p1^e1 * p2^e2 * ... * pk^ek, then the total
    number of divisors is (e1+1)*(e2+1)*...*(ek+1)
ll numDiv(ll N) {
    auto factors = primeFactors(N);
    ll ans = 1;
    for (auto &f : factors)
```



```

    ans *= (f.second + 1);
    return ans;
}

// sumDiv(N): Sum the divisors of N.
// If N = p1^e1 * p2^e2 * ... * pk^ek, then
// sumDiv(N) = ((p1^(e1+1)-1)/(p1-1)) *
// ((p2^(e2+1)-1)/(p2-1)) * ... * ((pk^(ek+1)-1)/(pk-1))
ll sumDiv(ll N) {
    auto factors = primeFactors(N);
    ll ans = 1;
    for (auto &f : factors) {
        ll p = f.first, exp = f.second;
        ll sum = 0;
        ll term = 1;
        for (int i = 0; i <= exp; i++){
            sum += term;
            term *= p;
        }
        ans *= sum;
    }
    return ans;
}

// EulerPhi(N): Calculate the Euler's Totient function of
// N.
// EulerPhi(N) = N * product_{p | N} (1 - 1/p)
ll EulerPhi(ll N) {
    ll ans = N;
    for (int i = 0; i < (int)primes.size() &&
        (ll)primes[i] * primes[i] <= N; ++i) {
        if (N % primes[i] == 0) {
            ans -= ans / primes[i]; // subtract multiples
            // of the prime.
            while (N % primes[i] == 0)
                N /= primes[i];
        }
    }
    if (N != 1) ans -= ans / N; // if there's any prime
    // factor left.
    return ans;
}

// segsie(L, R): Returns a vector of prime numbers in the
// interval [L, R].
// This function is useful when R is too large to
// generate a complete sieve.
vector<ll> segsie(ll L, ll R) {
    vector<ll> segment;
    vector<bool> mark(R - L + 1, true);
    if (L == 1) mark[0] = false; // 1 is not prime.

    // Use primes from the basic sieve to mark non-primes
    // in [L, R].
    for (auto p : primes) {
        if ((ll)p * p > R) break;
        // Find the minimum multiple of p in the [L, R]
        // range.
        ll start = max((ll)p * p, ((L + p - 1) / p) * p);
        for (ll j = start; j <= R; j += p)
            mark[j - L] = false;
    }

    for (ll i = 0; i < (ll)mark.size(); i++) {
        if (mark[i])
            segment.push_back(i + L);
    }
    return segment;
}

```

3 Tamzid

3.1 StressTest

3.1.1 checker

```

#include <bits/stdc++.h>

typedef long long int ll;
using namespace std;

int n;

ll a,b,c,d;

int readAns(istream &fin)
{
    ll ans;
    fin>>ans;

    if(ans==-1)return ans;

    if(((ans|b)-(ans&c))==d)
    {
        return 1;
    }

    return 0;
}

int main(int argc, char *argv[])
{
    ifstream fin("input file",ifstream::in);
    ifstream ans("myAnswer",ifstream::in);
    ifstream cor("correctAnswer",ifstream::in);

    fin>>n;
    fin>>b>>c>>d;

    int myAnswer=readAns(ans);
    int correctAnswer=readAns(cor);

    if(myAnswer==-1 && correctAnswer!=-1)
    {
        cout<<"Brute force found a solution, but the code
        // didn't!!"<<endl;
        return -1;
    }
    if(myAnswer!=correctAnswer)
    {
        cout<<"Wrong Answer: ";
        return -1;
    }

    return 0;
}

```

3.1.2 gen

```

#include <bits/stdc++.h>
using namespace std;
typedef long long int ll;

int rnd(int a, int b)
{
    return a + rand() % (b - a + 1);
}

int main(int argc, char *argv[])
{
    int seed = atoi(argv[1]);
    srand(seed);
}

```

```

int n = rnd(1, 500);
cout << n <<endl;
}

```

3.1.3 script

```

# script.sh

set -e
g++ code.cpp -o code
g++ gen.cpp -o gen
g++ brute.cpp -o brute
for((i = 1; ; ++i)); do
    ./gen $i > input file
    ./code < input file > myAnswer
    ./brute < input file > correctAnswer
    diff -Z myAnswer correctAnswer > /dev/null || break
    echo "Passed test: " $i
done
echo "WA on the following test:"
cat input file
echo "Your answer is:"
cat myAnswer
echo "Correct answer is:"
cat correctAnswer

```

3.2 KMP

```

void PrefixTable(string need, vector<ll>&preftable)
{
    ll siz = need.size();

    preftable[0] = -1;

    ll start = -1;
    for (ll i = 0; i < siz; i++)
    {
        while (start >= 0 && need[i] != need[start])
            start = preftable[start];
        start++;
        preftable[i+1] = start;
        // cout<<preftable[i+1]<<" ";
    }
}

void KMPSearch(string given, string
    // need,vector<ll>&PrefixTable)
{
    ll start1=0,start2=0,siz1=given.size(),siz2=need.siz
    // e();

    vector<ll>matchingindex;

    while(start1<siz1)
    {
        while(start2>=0 && given[start1]!=need[start2])
            start2=PrefixTable[start2];
        start1++,start2++;
        if(start2==siz2)
        {
            matchingindex.push_back(start1-start2+1);
            start2=PrefixTable[start2];
        }
    }

    cout<<matchingindex.size()<<endl;
    // for(auto it:matchingindex)cout<<it<<" ";
}

```

3.3 area_intersectingpoint

```

ld area(ld ax, ld ay, ld bx, ld by, ld cx, ld cy)
{
    ld sidea = (ax - bx) * (ax - bx) + (ay - by) * (ay - by);
    sidea = sqrt(sidea);
    ld sideb = (cx - bx) * (cx - bx) + (cy - by) * (cy - by);
    sideb = sqrt(sideb);
    ld sidec = (ax - cx) * (ax - cx) + (ay - cy) * (ay - cy);
    sidec = sqrt(sidec);
    ld perimeter = (sidea + sideb + sidec) / 2.0;
    ld area = perimeter * (perimeter - sidea) *
        (perimeter - sideb) * (perimeter - sidec);
    area = sqrt(area);
    return area;
}

pair<ld, ld> intersecting_point(ld x1, ld y1, ld x2, ld y2, ld x3, ld y3, ld x4, ld y4)
{
    pair<ld, ld> ret;
    auto &[x, y] = ret;

    x = (x1 * y2 - y1 * x2) * (x3 - x4) - (x1 - x2) *
        (x3 * y4 - y3 * x4);
    ld denominator = (x1 - x2) * (y3 - y4) - (y1 - y2) *
        (x3 - x4);
    if (!denominator)
        x = 0;
    else
        x /= denominator;

    y = (x1 * y2 - y1 * x2) * (y3 - y4) - (y1 - y2) *
        (x3 * y4 - y3 * x4);
    denominator = (x1 - x2) * (y3 - y4) - (y1 - y2) *
        (x3 - x4);
    if (!denominator)
        y = 0;
    else
        y /= denominator;
    return ret;
}

```

3.4 bellmanford

```

struct Edge
{
    ll from, to, cost;
};

ll vertex, edges; // no. of vertex and edges in the graph
vector<Edge> vec(200005); // store the edges from, to and cost.
vector<ll> dist(200005, INF);
vector<ll> parent(200005, -1);

void Bellman_Ford()
{
    dist[1] = 0;
    for (ll i = 1; i < vertex; i++)
    {

```

```

        for (Edge it : vec)
        {
            if (dist[it.from] < INF)
            {
                if (dist[it.from] + it.cost < dist[it.to])
                {
                    dist[it.to] = max(dist[it.from] + it.cost, -INF);
                    parent[it.to] = it.from;
                }
            }
        }
    }

    void retrieve_shortest_path(vector<ll> &path)
    {
        if (dist[vertex] == INF)
            return; // NO Path Exist

        for (ll current = vertex; current != -1; current = parent[current])
        {
            path.push_back(current);
        }
        reverse(path.begin(), path.end());
    }

    void find_negative_cycle(vector<ll> &path)
    {
        // run this after bellman ford has been run
        ll start = -1;
        for (Edge it : vec)
        {
            if (dist[it.from] < INF)
            {
                if (dist[it.from] + it.cost < dist[it.to])
                {
                    dist[it.to] = dist[it.from] + it.cost;
                    start = it.to;
                    parent[it.to] = it.from;
                }
            }
        }

        if (start == -1)
            return; // no Negative cycle found so, path is empty

        for (ll times = 0; times < vertex; times++)
            start = parent[start];

        path.push_back(start);
        for (ll current = parent[start]; current != parent[current]; current = parent[current])
        {
            path.push_back(current);
            if (current == start) break;
        }
        reverse(path.begin(), path.end());
    }
}

```

3.5 convexhull

```

struct Point
{
    int64_t x, y; // x*x or y*y should not overflow
    Point() {}
    Point(int64_t x, int64_t y) : x(x), y(y) {}
}

```

```

inline bool operator<(const Point &p) const
{
    return ((x < p.x) || (x == p.x && y < p.y));
};

int64_t cross(const Point &O, const Point &A, const Point &B)
{
    return ((A.x - O.x) * (B.y - O.y)) - ((A.y - O.y) * (B.x - O.x));
}

vector<Point> get_convex_hull(vector<Point> P)
{
    int n = P.size(), k = 0;
    vector<Point> H(n <= 1);
    sort(P.begin(), P.end());
    for (int i = 0; i < n; i++)
    {
        while (k >= 2 && cross(H[k - 2], H[k - 1], P[i]) <= 0)
            k--;
        H[k++] = P[i];
    }
    for (int i = n - 2, t = k + 1; i >= 0; i--)
    {
        while (k >= t && cross(H[k - 2], H[k - 1], P[i]) <= 0)
            k--;
        H[k++] = P[i];
    }
    H.resize(k - 1);
    return H;
}

/**
 * Returns whether the polygon is convex or not
 * Points in P is given in clock-wise or anti-clockwise order
 */
bool is_convex(vector<Point> &P)
{
    int n = P.size();
    if (n <= 2)
        return false; // Line or point is not convex

    n++, P.push_back(P[0]); // Last point = first point
    bool flag1 = (cross(P[0], P[1], P[2]) > 0);
    for (int i = 1; (i + 1) < n; i++)
    {
        bool flag2 = (cross(P[i], P[i + 1], P[i + 2]) > 0);
        if (flag1 ^ flag2)
            return false;
    }
    return true;
}

ll dotproduct(Point O, Point A, Point B)
{
    return (A.x - O.x) * (B.x - O.x) + (A.y - O.y) * (B.y - O.y);
}

```

```

}

double get_angle(const Point &O, const Point &A, const
    Point &B)
{
    ll product = dotproduct(O, A, B);
    ll x1 = A.x - O.x, y1 = A.y - O.y;
    ll x2 = B.x - O.x, y2 = B.y - O.y;

    double v1 = sqrt(x1 * x1 + y1 * y1);
    double v2 = sqrt(x2 * x2 + y2 * y2);

    double cos_theta = product / (v1 * v2);

    double angle = acos(cos_theta);
    angle /= pi;
    angle *= 180.0;

    return angle;
}

```

3.6 moalgo

```

struct MO
{
    struct node
    {
        ll id, left, right;
    };

    vector<node> queries;
    vector<ll> val, cnt;
    vector<ll> ans;

    ll num, questions, block;

    void build(vector<ll> &vec, vector<pair<ll, ll>> &q)
    {
        num = vec.size();
        val.resize(num + 1);

        block = sqrt(num);
        // 1-based indexing
        for (ll i = 1; i <= num; i++)
            val[i] = vec[i - 1];

        questions = q.size();
        queries.resize(questions);
        ans.resize(questions);

        cnt = vec;
        sort(cnt.begin(), cnt.end());
        cnt.erase(unique(cnt.begin(), cnt.end()),
            cnt.end());
        map<ll, ll> mp;
        for (ll i = 0; i < cnt.size(); i++)
            mp[cnt[i]] = i;

        for (ll i = 1; i <= num; i++)
            val[i] = mp[val[i]];

        cnt.clear();
        cnt.resize(mp.size());

        for (ll i = 0; i < questions; i++)
        {
            queries[i].id = i;
            queries[i].left = q[i].first;
            queries[i].right = q[i].second;
        }
    }
}

```

```

sort(queries.begin(), queries.end(), [&](node
    & one, node two)
    {
        ll start1 = one.left / block, start2 =
            two.left / block;
        ll end1 = one.right / block, end2 =
            two.right / block;

        if(start1==start2) return end1<end2;
        return start1<start2; });

    }

    ll times = 0;

    void add(ll ind)
    {
        if (!cnt[ind])
            times++;

        cnt[ind]++;
    }

    void del(ll ind)
    {
        if (cnt[ind] == 1)
            times--;

        cnt[ind]--;
    }

    vector<ll> getresult()
    {
        ll start = 1, end = 1;
        add(val[1]);

        for (ll i = 0; i < questions; i++)
        {
            ll left = queries[i].left, right =
                queries[i].right;

            while (end < right)
                add(val[++end]);
            while (start > left)
                add(val[--start]);

            while (end > right)
                del(val[end--]);
            while (start < left)
                del(val[start++]);

            ans[queries[i].id] = times;
        }

        return ans;
    }
};

```

3.7 ncr

```

ll factorial[1000006];

void precompute()
{
    factorial[0] = factorial[1] = 1;
    for (ll i = 2; i <= 1000000; i++)
        factorial[i] = (factorial[i - 1] * i) % modular;
}

ll modularBinaryExponentiation(ll base, ll exponent)
{
    if (exponent == 0)

```

```

        return 1;
    ll result = modularBinaryExponentiation(base,
        exponent / 2);
    if (exponent & 1)
        return (((result * result) % modular) * base)
            % modular;
    else
        return ((result * result) % modular);
}

ll ncr(ll n, ll r)
{
    if(r>n) return 0;

    ll ret=factorial[n];
    ret=(ret*modularBinaryExponentiation(factorial[r],mo
        dular-2))%modular;
    ret=(ret*modularBinaryExponentiation(factorial[n-r],
        modular-2))%modular;

    return ret;
}

```

3.8 polyhash

```

#define MAXLEN 1000006

constexpr uint64_t mod = (1ULL << 61) - 1;

const uint64_t seed = chrono::system_clock::now().time_s
   ince_epoch().count();
const uint64_t base = mt19937_64(seed)() % (mod / 3) +
    (mod / 3);

uint64_t base_pow[MAXLEN];

int64_t modmul(uint64_t a, uint64_t b)
{
    uint64_t l1 = (uint32_t)a, h1 = a >> 32, l2 =
        (uint32_t)b, h2 = b >> 32;
    uint64_t l = l1 * l2, m = l1 * h2 + l2 * h1, h = h1
        * h2;
    uint64_t ret = (l & mod) + (l >> 61) + (h << 3) + (m
        >> 29) + (m << 35 >> 3) + 1;
    ret = (ret & mod) + (ret >> 61);
    ret = (ret & mod) + (ret >> 61);
    return ret - 1;
}

void init()
{
    base_pow[0] = 1;
    for (int i = 1; i < MAXLEN; i++)
    {
        base_pow[i] = modmul(base_pow[i - 1], base);
    }
}

struct PolyHash
{
    // Remove suff vector and usage if reverse hash is
    // not required for more speed
    vector<int64_t> pref, suff;

    PolyHash() {}

    template <typename T>
    PolyHash(const vector<T> &ar)
    {

```



```

if (!base_pow[0])
    init();

int n = ar.size();
assert(n < MAXLEN);
pref.resize(n + 3, 0), suff.resize(n + 3, 0);

for (int i = 1; i <= n; i++)
{
    pref[i] = modmul(pref[i - 1], base) + ar[i - 1] + 997;
    if (pref[i] >= mod)
        pref[i] -= mod;
}

// for (int i = n; i >= 1; i--)
// {
//     suff[i] = modmul(suff[i + 1], base) + ar[i - 1] + 997;
//     if (suff[i] >= mod)
//         suff[i] -= mod;
// }

PolyHash(string &str)
: PolyHash(vector<char>(str.begin(), str.end()))
{}

uint64_t get_hash(int l, int r)
{
    int64_t h = pref[r + 1] - modmul(base_pow[r - l + 1], pref[l]);
    return h < 0 ? h + mod : h;
}

uint64_t rev_hash(int l, int r)
{
    int64_t h = suff[l + 1] - modmul(base_pow[r - l + 1], suff[r + 2]);
    return h < 0 ? h + mod : h;
}
};

```

3.9 seckosaraju

```

ll num, edges;
vector<vector<ll>> adj, reverseadj;
vector<ll>visited, visitororder, ans;
vector<vector<ll>> allscc;

```

```

void dfs(ll node)
{
    visited[node]=1;
    for(auto it:adj[node])
    {
        if(!visited[it])dfs(it);
    }
    visitororder.push_back(node);
}

void dfs2(ll node,ll times)
{
    visited[node]=1;
    ans[node]=times;
    allscc[times-1].push_back(node);
    for(auto it:reverseadj[node])
    {
        if(!visited[it])
            dfs2(it,times);
    }
}

```

```

void kosaraju()
{
    for(ll i=0;i<num;i++)
    {
        if(!visited[i])dfs(i);
    }

    reverse(visitororder.begin(),visitororder.end());
    visited.assign(num,0);
    ll scc=0;
    for(auto i:visitororder)
    {
        if(!visited[i])
        {
            dfs2(i,++scc);
        }
    }

    cout<<scc<<endl;
    for(auto it:ans)cout<<it<<" ";

    for(ll i=0;i<scc;i++)
    {
        cout<<i<<": ";
        for(auto it:allscc[i])cout<<it<<" ";cout<<endl;
    }
}

```

3.10 sparsetable

```

struct sparsetable
{
    vector<vector<ll>> lookup;
    vector<ll> arr;
    ll times = 1, siz;

    void initialize(ll num)
    {
        siz = num;
        arr.resize(num);
        times = log2(num);
        lookup.resize(num, vector<ll>(times + 1));
    }

    ll merge(ll val1, ll val2)
    {
        return min(val1,val2);
    }

    void build(vector<ll> &vec)
    {
        for (ll i = 0; i < siz; i++)
            lookup[i][0] = vec[i];

        for (ll j = 1; j <= times; j++)
        {
            for (ll i = 0; i + (1LL << j) <= siz; i++)
                lookup[i][j] = merge(lookup[i][j - 1],
                    lookup[i + (1LL << (j - 1))][j - 1]);
        }
    }

    ll query(ll lo, ll hi)
    {
        ll j = log2(hi - lo + 1);
        return merge(lookup[lo][j], lookup[hi - (1LL << j) + 1][j]);
    }
};

```

3.11 tree

```

vector<vector<ll>> adj;
vector<ll> depth;
vector<vector<ll>> parent;

void dfs(ll node, ll par, ll deep)
{
    parent[0][node] = par;
    depth[node] = deep;
    for (ll i = 1; i < 30; i++)
    {
        parent[i][node] = parent[i - 1][parent[i-1][node]];
    }
    for (auto it : adj[node])
    {
        if (it != par)
        {
            dfs(it, node, deep + 1);
        }
    }
}

ll lca(ll u, ll v)
{
    if (depth[u] < depth[v])
        swap(u, v);
    ll diff = depth[u] - depth[v];
    for (ll i = 0; i < 25; i++)
    {
        if ((diff >> i) & 1)
            u = parent[i][u];
    }
    if (u == v)
        return u;
    for (ll i = 25; i >= 0; i--)
    {
        if (parent[i][u] != parent[i][v])
        {
            u = parent[i][u];
            v = parent[i][v];
        }
    }
    return parent[0][u];
}

ll distance(ll u, ll v)
{
    ll l = lca(u, v);
    ll du = depth[u] - depth[l];
    ll dv = depth[v] - depth[l];
    return du + dv;
}

```

3.12 treediameter

```

ll num,ans;
vector<vector<ll>>adj;
vector<bool>visited;
ll find_diameter(ll node)
{
    visited[node]=1;
    multiset<ll>ms; ms.insert(0);
    for(auto it:adj[node])
    {
        if(!visited[it])
        {
            ms.insert(find_diameter(it));

```

```

    }
}
auto it=ms.rbegin();
ll maxi=*it;    it--;
ll second=*it;
ans=max(ans,maxi+second);
return maxi+1;
}

```

4 zFrom Sifat Bhai

4.1 Data Structure

4.1.1 BIT_range_update

```

struct BIT {
    long long M[N], A[N];
    BIT() {
        memset(M, 0, sizeof M);
        memset(A, 0, sizeof A);
    }
    void update(int i, long long mul, long long add) {
        while (i < N) {
            M[i] += mul;
            A[i] += add;
            i |= (i + 1);
        }
    }
    void upd(int l, int r, long long x) {
        update(l, x, -x * (l - 1));
        update(r, -x, x * r);
    }
    long long query(int i) {
        long long mul = 0, add = 0;
        int st = i;
        while (i >= 0) {
            mul += M[i];
            add += A[i];
            i = (i & (i + 1)) - 1;
        }
        return (mul * st + add);
    }
    long long query(int l, int r) {
        return query(r) - query(l - 1);
    }
} t;

```

4.1.2 CHT

```

/**
 * Lines should be added non-increasing order of m for
 * - minimizing
 * Non-decreasing order of m for maximizing
 * Intersection point of two lines (m1,c1) , (m2,c2) is
 * x = (c2-c1)/(m1-m2)
 */

```

```

ll M[MAX], C[MAX];

struct CHT {
    int len, cur;
    void init() {
        len = 0, cur = 0;
    }

    /// returns true if line[len-1] is unnecessary when we
    /// add line(nm,nc)
    inline bool isBad(ll nm, ll nc) {
        return (
            (C[len-1]-C[len-2])/(double)(M[len-2]-M[len-1])
            <= (nc-C[len-2])/(double)(M[len-2]-nm) );
    }
}

```

```

//return ( (C[len-1]-C[len-2])*(M[len-2]-nm) >=
// (M[len-2]-M[len-1])*(nc-C[len-2]) );
}

inline void addLine(ll nm, ll nc) {
    if(len == 0) M[len] = nm, C[len] = nc, ++len;
    else if (M[len-1] == nm) {
        if(C[len-1] <= nc) return; /// <= to minimize, >=
        to maximize
        else C[len-1] = nc;
    }
    else {
        while(len >= 2 && isBad(nm,nc)) --len;
        M[len] = nm, C[len] = nc, ++len;
    }
}

inline ll getY(int id, ll x) {
    return (M[id]*x + C[id]);
}

inline ll sortedQuery(ll x) {
    if(cur >= len) cur = len-1;
    while( cur < len-1 && getY(cur+1,x) >= getY(cur,x) )
        cur++; /// <= to minimize, >= to maximize
    return getY(cur,x);
}

inline ll TS(ll x) {
    int low = 0, high = len-1, mid;
    while( high - low > 1 ) {
        mid = low + high >> 1;
        if(getY(mid,x) < getY(mid+1,x)) low = mid + 1; ///
        > to minimize, < to maximize
        else high = mid;
    }
    return max(getY(low,x), getY(high,x)); /// adjust
    to min/max
}

}
}

CHT cht;
cht.init();

```

4.1.3 dynamic_cht

```

//add lines with -m and -b and return -ans to
//make this code work for minimums.(not -x)
const ll inf = -(1LL << 62);
struct line {
    ll m, b;
    mutable function<const line*()> succ;
    bool operator < (const line& rhs) const {
        if (rhs.b != inf) return m < rhs.m;
        const line* s = succ();
        if (!s) return 0;
        ll x = rhs.m;
        return b - s->b < (s->m - m) * x;
    }
}

struct CHT : public multiset<line> {
    bool bad(iterator y) {
        auto z = next(y);
        if (y == begin()) {
            if (z == end()) return 0;
            return y->m == z->m && y->b <= z->b;
        }
        auto x = prev(y);
        if (z == end()) return y->m == x->m && y->b <=
        x->b;
    }
}

```

```

return 1.0 * (x -> b - y -> b) * (z -> m - y -> m)
- >= 1.0 * (y -> b - z -> b) * (y -> m - x -> m);
}

void add(ll m, ll b) {
    auto y = insert({m, b});
    y->succ = [=] { return next(y) == end() ? 0 :
    &*next(y); };
    if (bad(y)) {
        erase(y);
        return;
    }
    while (next(y) != end() && bad(next(y)))
        erase(next(y));
    while (y != begin() && bad(prev(y))) erase(prev(y));
}

ll query(ll x) {
    auto l = *lower_bound((line) {
        x, inf
    });
    return l.m * x + l.b;
}

};

CHT* cht;
ll a[N], b[N];
int32_t main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);

    int n;
    cin >> n;
    for(int i = 0; i < n; i++) cin >> a[i];
    for(int i = 0; i < n; i++) cin >> b[i];
    cht = new CHT();
    cht -> add(-b[0], 0);
    ll ans = 0;
    for(int i = 1; i < n; i++) {
        ans = -cht -> query(a[i]);
        cht -> add(-b[i], -ans);
    }
    cout << ans << nl;
    return 0;
}

```

4.1.4 gp_hash_table

```

#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
const int RANDOM = chrono::high_resolution_clock::now().
- time_since_epoch().count();
struct chash {
    int operator()(int x) const { return x ^ RANDOM; }
};
gp_hash_table<key, int, chash> table;

```

4.1.5 persistant_segtree

```

/** Persistent Segment Tree using static Array
Point Update, Range Sum
Initialize ncnt to 0 in every test case */

const int MAX = 100010;
int ncnt = 0;

struct node {
    int sum;
    int left, right;
}

```

```

node() {}
node(int val) {
    sum = val;
    left = right = -1;
}
} tree[ ? ];

/// input array
int ara[MAX];
/// root nodes for all versions
int version[MAX];

void build(int n,int st,int ed) {
    if (st==ed) {
        tree[n] = node(ara[st]);
        return;
    }

    int mid = (st+ed) / 2;

    tree[n].left = ++ncnt;
    tree[n].right = ++ncnt;

    build(tree[n].left, st, mid);
    build(tree[n].right, mid+1, ed);

    tree[n].sum = tree[tree[n].left].sum +
        tree[tree[n].right].sum;
}

void update(int prev,int cur,int st,int ed,int id, int
    val)
{
    if (id > ed or id < st) return;
    if (st == ed) {
        tree[cur] = node(val);
        return;
    }
    int mid = (st+ed) / 2;
    if (id <= mid) {
        tree[cur].right = tree[prev].right;
        tree[cur].left = ++ncnt;
        update(tree[prev].left,tree[cur].left, st, mid, id,
            val);
    }
    else {
        tree[cur].left = tree[prev].left;
        tree[cur].right = ++ncnt;
        update(tree[prev].right, tree[cur].right, mid+1, ed,
            id, val);
    }
    tree[cur].sum = tree[tree[cur].left].sum +
        tree[tree[cur].right].sum;
}

int query(int n,int st,int ed,int i,int j){
    if(st>=i && ed<=j) return tree[n].sum;
    int mid = (st+ed)/2;
    if(mid<i) return query(tree[n].right,mid+1,ed,i,j);
    else if(mid>=j) return query(tree[n].left,st,mid,i,j);
    else return query(tree[n].left,st,mid,i,j) +
        query(tree[n].right,mid+1,ed,i,j);
}

int main() {
    int n,q,l,r,k;

    sii(n,q);

    version[0] = ++ncnt;
    build(version[0],1,n);

    version[1] = ++ncnt;

```

```

update(version[0],version[1],1,n,id,val);

query(version[0],1,n,id,id);
query(version[1],1,n,id,id);

return 0;
}

4.2 Geometry
4.2.1 2D Primitive
4.2.1.1 Angle A class for ordering angles (as represented by
int points and a number of rotations around the origin). Useful
for rotational sweeping. Sometimes also represents points or
vectors.

/* Usage:
 * vector<Angle> v = {w[0], w[0].t360() ...}; // sorted
 * int j = 0; rep(i,0,n) { while (v[j] < v[i].t180())
 * ++j; }
 * // sweeps j such that (j-i) represents the number of
 * positively oriented triangles with vertices at 0 and
 * i
 */
struct Angle {
    int x, y;
    int t;
    Angle(int x, int y, int t=0) : x(x), y(y), t(t) {}
    Angle operator-(Angle b) const { return {x-b.x, y-b.y,
        t}; }
    int half() const {
        assert(x || y);
        return y < 0 || (y == 0 && x < 0);
    }
    Angle t90() const { return {-y, x, t + (half() && x >=
        0)}; }
    Angle t180() const { return {-x, -y, t + half()}; }
    Angle t360() const { return {x, y, t + 1}; }
};
bool operator<(Angle a, Angle b) {
    // add a.dist2() and b.dist2() to also compare
    // distances
    return make_tuple(a.t, a.half(), a.y * (ll)b.x) <
        make_tuple(b.t, b.half(), a.x * (ll)b.y);
}

// Given two points, this calculates the smallest angle
// between
// them, i.e., the angle that covers the defined line
// segment.
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
    if (b < a) swap(a, b);
    return (b < a.t180() ? make_pair(a, b) : make_pair(b,
        a.t360()));
}
Angle operator+(Angle a, Angle b) { // point a + vector b
    Angle r(a.x + b.x, a.y + b.y, a.t);
    if (a.t180() < r) r.t--;
    return r.t180() < a ? r.t360() : r;
}
Angle angleDiff(Angle a, Angle b) { // angle b - angle a
    int tu = b.t - a.t; a.t = b.t;
    return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x, tu - (b
        < a)};
}

```

4.2.1.2 On Segment

```

/* Description: Returns true iff p lies on the line
 * segment from s to e.
 * Use \texttt{(segDist(s,e,p)<=epsilon)} instead when
 * using Point<double>.
 */
#include "Point.h"

```

```

template<class P> bool onSegment(P s, P e, P p) {
    return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}

```

4.2.1.3 Point Sort

```

// sort the points in counterclockwise order that starts
// from the half line x0,y=0.
#include <bits/stdc++.h>

```

```

using namespace std;

typedef long long ll;
typedef pair<ll, ll> point;

#define x first
#define y second

```

```

int main() {
    int n; cin >> n;
    vector<point> p(n);
    for (auto &it : p) scanf("%lld %lld", &it.x, &it.y);
    sort(p.begin(), p.end(), [] (point a, point b) {
        return atan2l(a.y, a.x) < atan2l(b.y, b.x);
    });
    for (auto it : p) printf("%lld %lld\n", it.x, it.y);
    return 0;
}

```

4.2.1.4 Point

```

// Class to handle points in the plane. T can be e.g.
// double or long long. (Avoid int.)
template<class T> int sgn(T x) { return (x > 0) - (x <
    0); }
template<class T>
struct Point {
    typedef Point P;
    T x, y;
    explicit Point(T x=0, T y=0) : x(x), y(y) {}
    bool operator<(P p) const { return tie(x,y) <
        tie(p.x,p.y); }
    bool operator==(P p) const { return
        tie(x,y)==tie(p.x,p.y); }
    P operator+(P p) const { return P(x+p.x, y+p.y); }
    P operator-(P p) const { return P(x-p.x, y-p.y); }
    P operator*(T d) const { return P(x*d, y*d); }
    P operator/(T d) const { return P(x/d, y/d); }
    T dot(P p) const { return x*p.x + y*p.y; }
    T cross(P p) const { return x*p.y - y*p.x; }
    T cross(P a, P b) const { return
        (a-*this).cross(b-*this); }
    T dist2() const { return x*x + y*y; }
}

```

```
double dist() const { return sqrt((double)dist2()); }
// angle to x-axis in interval [-pi, pi]
double angle() const { return atan2(y, x); }
P unit() const { return *this/dist(); } // makes
    ← dist()=1
P perp() const { return P(-y, x); } // rotates +90
    ← degrees
P normal() const { return perp().unit(); }
// returns point rotated 'a' radians ccw around the
    ← origin
P rotate(double a) const {
    return P(x*cos(a)-y*sin(a), x*sin(a)+y*cos(a)); }
friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << ", " << p.y << ")"; }
};
```

4.2.1.5 Side Of Returns where p is as seen from s towards e . $1/0/-1 \Leftrightarrow$ left/on line/right. If the optional argument eps is given 0 is returned if p is within distance eps from the line. P is supposed to be $\text{Point}<T>$ where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

```
/* Usage:
 * bool left = sideOf(p1,p2,q)==1;
 * Status: tested
 */
#include "Point.h"

template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }

template<class P>
int sideOf(const P& s, const P& e, const P& p, double
    ← eps) {
    auto a = (e-s).cross(p-s);
    double l = (e-s).dist()*eps;
    return (a > l) - (a < -l);
}
```

4.2.2 3D

4.2.2.1 3D Convex Hull

```
#include <bits/stdc++.h>

#define ll long long
#define sz(x) ((int) (x).size())
#define all(x) (x).begin(), (x).end()
#define vi vector<int>
#define pii pair<int, int>
#define rep(i, a, b) for(int i = (a); i < (b); i++)
using namespace std;
template<typename T>
using minpq = priority_queue<T, vector<T>, greater<T>>;

typedef long double ftype;

struct pt3 {
    ftype x, y, z;
    pt3(ftype x = 0, ftype y = 0, ftype z = 0) : x(x),
        ← y(y), z(z) {}
    pt3 operator-(const pt3 &o) const {
        return pt3(x - o.x, y - o.y, z - o.z);
    }
    pt3 cross(const pt3 &o) const {
```

```
    return pt3(y * o.z - z * o.y, z * o.x - x * o.z, x *
        ← o.y - y * o.x);
}
ftype dot(const pt3 &o) const {
    return x * o.x + y * o.y + z * o.z;
}
};

// A face is represented by the indices of its three
    ← points a, b, c.
// It also stores an outward-facing normal vector q
struct face {
    int a, b, c;
    pt3 q;
};

// modify this depending on the coordinate sizes in your
    ← use case
const ftype EPS = 1e-9;

vector<face> hull3(const vector<pt3> &p) {
    int n = sz(p);
    assert(n >= 3);
    vector<face> f;

    // Consider an edge (a->b) dead if it is not a CCW edge
    ← of some current face
    // If an edge is alive but not its reverse, this is an
    ← exposed edge.
    // We should add new faces on the exposed edges.
    vector<vector<bool>> dead(n, vector<bool>(n, true));
    auto add face = [&](int a, int b, int c) {
        f.push_back({a, b, c, (p[b] - p[a]).cross(p[c] -
            ← p[a])});
        dead[a][b] = dead[b][c] = dead[c][a] = false;
    };

    // Initialize the convex hull of the first 3 points as
    ← a
    // triangular disk with two faces of opposite
    ← orientation
    add face(0, 1, 2);
    add face(0, 2, 1);
    rep(i, 3, n) {
        // f2 will be the list of faces invisible to the
            ← added point p[i]
        vector<face> f2;
        for(face &F : f) {
            if((p[i] - p[F.a]).dot(F.q) > EPS) {
                // this face is visible to the new point, so mark
                    ← its edges as dead
                dead[F.a][F.b] = dead[F.b][F.c] = dead[F.c][F.a]
                    ← = true;
            } else {
                f2.push_back(F);
            }
        }
        // Add a new face for each exposed edge.
        // Only check edges of alive faces for being exposed.
        f.clear();
        for(face &F : f2) {
            int arr[3] = {F.a, F.b, F.c};
            rep(j, 0, 3) {
                int a = arr[j], b = arr[(j + 1) % 3];
                if(dead[b][a]) {
                    add_face(b, a, i);
                }
            }
        }
        f.insert(f.end(), all(f2));
    }
}
```

```
    return f;
}
```

4.2.2.2 Point3D Class to handle points in 3D space. T can be e.g. double or long long.

```
template<class T> struct Point3D {
    typedef Point3D P;
    typedef const P& R;
    T x, y, z;
    explicit Point3D(T x=0, T y=0, T z=0) : x(x), y(y),
        ← z(z) {}
    bool operator<(R p) const {
        return tie(x, y, z) < tie(p.x, p.y, p.z); }
    bool operator==(R p) const {
        return tie(x, y, z) == tie(p.x, p.y, p.z); }
    P operator+(R p) const { return P(x+p.x, y+p.y,
        ← z+p.z); }
    P operator-(R p) const { return P(x-p.x, y-p.y,
        ← z-p.z); }
    P operator*(T d) const { return P(x*d, y*d, z*d); }
    P operator/(T d) const { return P(x/d, y/d, z/d); }
    T dot(R p) const { return x*p.x + y*p.y + z*p.z; }
    P cross(R p) const {
        return P(y*p.z - z*p.y, z*p.x - x*p.z, x*p.y -
            ← y*p.x);
    }
    T dist2() const { return x*x + y*y + z*z; }
    double dist() const { return sqrt((double)dist2()); }
    //Azimuthal angle (longitude) to x-axis in interval
        ← [-pi, pi]
    double phi() const { return atan2(y, x); }
    //Zenith angle (latitude) to the z-axis in interval [0,
        ← pi]
    double theta() const { return atan2(sqrt(x*x+y*y), z); }
    P unit() const { return *this/(T)dist(); } //makes
        ← dist()=1
    //returns unit vector normal to *this and p
    P normal(P p) const { return cross(p).unit(); }
    //returns point rotated 'angle' radians ccw around axis
    P rotate(double angle, P axis) const {
        double s = sin(angle), c = cos(angle); P u =
            ← axis.unit();
        return u*dot(u)*(1-c) + (*this)*c - cross(u)*s;
    }
};
```

4.2.2.3 Polyhedron Volume Description: Magic formula for the volume of a polyhedron. Faces should point outwards.

```
template<class V, class L>
double signedPolyVolume(const V& p, const L& trilst) {
    double v = 0;
    for (auto i : trilst) v +=
        ← p[i.a].cross(p[i.b]).dot(p[i.c]);
    return v / 6;
}
```


4.2.2.4 Spherical Distance Returns the shortest distance on the sphere with radius `radius` between the points with azimuthal angles (longitude) `f1` (ϕ_1) and `f2` (ϕ_2) from x axis and zenith angles (latitude) `t1` (θ_1) and `t2` (θ_2) from z axis (0 = north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. `dx*radius` is then the difference between the two points in the x direction and `d*radius` is the total distance between the points.

```
double sphericalDistance(double f1, double t1,
    double f2, double t2, double radius) {
    double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
    double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
    double dz = cos(t2) - cos(t1);
    double d = sqrt(dx*dx + dy*dy + dz*dz);
    return radius*2*asin(d/2);
}
```

4.2.3 Circle

4.2.3.1 Circle Intersection Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

```
#include "Point.h"
typedef Point<double> P;
bool circleInter(P a, P b, double r1, double r2, pair<P, P>*
    out) {
    if (a == b) { assert(r1 != r2); return false; }
    P vec = b - a;
    double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
        p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 -
        p*p*d2;
    if (sum*sum < d2 || dif*dif > d2) return false;
    P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2)
        / d2);
    *out = {mid + per, mid - per};
    return true;
}
```

4.2.3.2 Circle Polygon Intersection Returns the area of the intersection of a circle with a ccw polygon. Time: $O(n)$

```
#include "Point.h"
typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
    auto tri = [&](P p, P q) {
        auto r2 = r * r / 2;
        P d = q - p;
        auto a = d.dot(p)/d.dist2(), b =
            (p.dist2()-r*r)/d.dist2();
        auto det = a * a - b;
        if (det <= 0) return arg(p, q) * r2;
        auto s = max(0., -a-sqrt(det)), t = min(1.,
            -a+sqrt(det));
        if (t < 0 || 1 <= s) return arg(p, q) * r2;
    };
    return arg(p, q) * r2;
}
```

```
P u = p + d * s, v = p + d * t;
return arg(p, u) * r2 + u.cross(v)/2 + arg(v, q) * r2;
};
auto sum = 0.0;
rep(i, 0, sz(ps))
    sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
return sum;
}
```

4.2.3.3 Circle Tangents Finds the external tangents of two circles, or internal if `r2` is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case `.first` = `.second` and the tangent line is perpendicular to the line between the centers). `.first` and `.second` give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set `r2` to 0.

```
#include "Point.h"
template<class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2,
    double r2) {
    P d = c2 - c1;
    double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
    if (d2 == 0 || h2 < 0) return {};
    vector<pair<P, P>> out;
    for (double sign : {-1, 1}) {
        P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
        out.push_back({c1 + v * r1, c2 + v * r2});
    }
    if (h2 == 0) out.pop_back();
    return out;
}
```

4.2.4 Polygon

4.2.4.1 Hull Diameter Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).

```
#include "Point.h"
typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
    int n = sz(S), j = n < 2 ? 0 : 1;
    pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
    rep(i, 0, j)
        for (; j = (j + 1) % n) {
            res = max(res, {(S[i] - S[j]).dist2(), {S[i],
                S[j]}});
            if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i])
                >= 0)
                break;
        }
    return res.second;
}
```

4.2.4.2 Line Hull Intersection Line-convex polygon intersection. The polygon must be ccw and have no collinear points.

`lineHull(line, poly)` returns a pair describing the intersection of a line with the polygon:

(-1, -1) if no collision,

(i, -1) if touching the corner *i*,

(i, i) if along side (i, i + 1),

(i, j) if crossing sides (i, i + 1) and (j, j + 1). In the last case, if a corner *i* is crossed, this is treated as happening on side (i, i + 1). The points are returned in the same order as the line hits the polygon. `extrVertex` returns the point of a hull with the max projection onto a line.

Time: $O(\log n)$

```
#include "Point.h"
#define cmp(i, j)
    sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) <
    0
template <class P> int extrVertex(vector<P>& poly, P
    dir) {
    int n = sz(poly), lo = 0, hi = n;
    if (extr(0)) return 0;
    while (lo + 1 < hi) {
        int m = (lo + hi) / 2;
        if (extr(m)) return m;
        int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
        (ls < ms || (ls == ms && ls == cmp(lo, m))) ? hi :
            lo = m;
    }
    return lo;
}
#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P>& poly) {
    int endA = extrVertex(poly, (a - b).perp());
    int endB = extrVertex(poly, (b - a).perp());
    if (cmpL(endA) < 0 || cmpL(endB) > 0)
        return {-1, -1};
    array<int, 2> res;
    rep(i, 0, 2) {
        int lo = endB, hi = endA, n = sz(poly);
        while ((lo + 1) % n != hi) {
            int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
            (cmpL(m) == cmpL(endB) ? lo : hi) = m;
        }
        res[i] = (lo + !cmpL(hi)) % n;
        swap(endA, endB);
    }
    if (res[0] == res[1]) return {res[0], -1};
    if (!cmpL(res[0]) && !cmpL(res[1]))
        switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly))
            {
                case 0: return {res[0], res[0]};
                case 2: return {res[1], res[1]};
            }
    return res;
}
```


4.2.4.3 Polygon Center

Returns the center of mass for a polygon.

Time: $O(n)$

```
#include "Point.h"
```

```
typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
    P res(0, 0); double A = 0;
    for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
        res = res + (v[i] + v[j]) * v[j].cross(v[i]);
        A += v[j].cross(v[i]);
    }
    return res / A / 3;
}
```

4.2.5 Minimum Enclosing Circle

// Expected runtime: $O(n)$

// Solves Gym 102299J

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
typedef long double ld;
typedef pair<ld, ld> point;
```

```
#define x first
#define y second
```

```
point operator + (const point &a, const point &b) {
    return point(a.x + b.x, a.y + b.y);
}
```

```
point operator - (const point &a, const point &b) {
    return point(a.x - b.x, a.y - b.y);
}
```

```
point operator * (const point &a, const ld &b) {
    return point(a.x * b, a.y * b);
}
```

```
point operator / (const point &a, const ld &b) {
    return point(a.x / b, a.y / b);
}
```

```
const ld EPS = 1e-8;
const ld INF = 1e20;
const ld PI = acos(-1);
```

```
inline ld dist (point a, point b) {
    return hypotl(a.x - b.x, a.y - b.y);
}
```

```
inline ld sqDist (point a, point b) {
    return (a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y);
}
```

```
inline ld dot (point a, point b) {
    return a.x * b.x + a.y * b.y;
}
```

```
inline ld cross (point a, point b) {
    return a.x * b.y - a.y * b.x;
}
```

```
inline ld cross (point a, point b, point c) {
    return cross(b - a, c - a);
}
```

```
inline point perp (point a) {
    return point(-a.y, a.x);
}

// circle through 3 points
pair<point, ld> getCircle (point a, point b, point c) {
    pair<point, ld> ret;
    ld den = (ld) 2 * cross(a, b, c);
    ret.x.x = ((c.y - a.y) * (dot(b, b) - dot(a, a)) -
        (b.y - a.y) * (dot(c, c) - dot(a, a))) / den;
    ret.x.y = ((b.x - a.x) * (dot(c, c) - dot(a, a)) -
        (c.x - a.x) * (dot(b, b) - dot(a, a))) / den;
    ret.y = dist(ret.x, a);
    return ret;
}

pair<point, ld> minCircleAux (vector<point> &s, point
    a, point b, int n) {
    ld lo = -INF, hi = INF;
    for (int i = 0; i < n; ++i) {
        auto si = cross(b - a, s[i] - a);
        if (fabs(si) < EPS) continue;
        point m = getCircle(a, b, s[i]).x;
        auto cr = cross(b - a, m - a);
        si < 0 ? hi = min(hi, cr) : lo = max(lo, cr);
    }
    ld v = 0 < lo ? lo : hi < 0 ? hi : 0;
    point c = (a + b) * 0.5 + perp(b - a) * v / sqDist(a,
        b);
    return {c, sqDist(a, c)};
}

pair<point, ld> minCircle (vector<point> &s, point a,
    int n) {
    random_shuffle(s.begin(), s.begin() + n);
    point b = s[0], c = (a + b) * 0.5;
    ld r = sqDist(a, c);
    for (int i = 1; i < n; ++i) {
        if (sqDist(s[i], c) > r * (1 + EPS)) {
            tie(c, r) = n == s.size() ? minCircle(s, s[i], i)
                : minCircleAux(s, a, s[i], i);
        }
    }
    return {c, r};
}

pair<point, ld> minCircle (vector<point> s) {
    assert(!s.empty());
    if (s.size() == 1) return {s[0], 0};
    return minCircle(s, s[0], s.size());
}

int n; vector<point> p;

int main() {
    cin >> n;
    while (n--) {
        double x, y;
        scanf("%lf %lf", &x, &y);
        p.emplace_back(x, y);
    }
    pair<point, ld> circ = minCircle(p);
    printf("%.12f %.12f %.12f\n", (double) circ.x.x,
        (double) circ.x.y, (double) (0.5 * circ.y));
    return 0;
}
```

4.2.6 Point In Polygon

```
// Test if a point is inside a convex polygon in  $O(\lg n)$ 
// time
// Solves SPOJ INOROUT
typedef long long ll;
typedef pair<ll, ll> point;

#define x first
#define y second

struct segment {
    point P1, P2;
    segment () {}
    segment (point P1, point P2) : P1(P1), P2(P2) {}
};

inline ll ccw (point A, point B, point C) {
    return (B.x - A.x) * (C.y - A.y) - (C.x - A.x) * (B.y - A.y);
}

inline bool pointOnSegment (segment S, point P) {
    ll x = P.x, y = P.y, x1 = S.P1.x, y1 = S.P1.y, x2 =
        S.P2.x, y2 = S.P2.y;
    ll a = x - x1, b = y - y1, c = x2 - x1, d = y2 - y1,
        dot = a * c + b * d, len = c * c + d * d;
    if (x1 == x2 and y1 == y2) return x1 == x and y1 == y;
    if (dot < 0 or dot > len) return 0;
    return x1 * len + dot * c == x * len and y1 * len +
        dot * d == y * len;
}

const int M = 17;
const int N = 10010;

struct polygon {
    int n; // n > 1
    point p[N]; // clockwise order
    polygon () {}
    polygon (int _n, point *T) {
        n = _n;
        for (int i = 0; i < n; ++i) p[i] = T[i];
    }

    bool contains (point P, bool strictlyInside) {
        int lo = 1, hi = n - 1;
        while (lo < hi) {
            int mid = lo + hi >> 1;
            if (ccw(p[0], P, p[mid]) > 0) lo = mid + 1;
            else hi = mid;
        }
        if (ccw(p[0], P, p[lo]) > 0) lo = 1;
        if (!strictlyInside and pointOnSegment(segment(p[0],
            p[n - 1]), P)) return 1;
        if (!strictlyInside and
            pointOnSegment(segment(p[lo], p[lo - 1]), P))
            return 1;
        if (lo == 1 or ccw(p[0], P, p[n - 1]) == 0) return 0;
        return ccw(p[lo], P, p[lo - 1]) < 0;
    }
};
```

4.2.7 sweep

```
const double EPS = 1E-9;
```

```
struct pt {
    double x, y;
```

```

};

struct seg {
    pt p, q;
    int id;

    double get_y(double x) const {
        if (abs(p.x - q.x) < EPS)
            return p.y;
        return p.y + (q.y - p.y) * (x - p.x) / (q.x - p.x);
    }
};

bool intersectld(double l1, double r1, double l2, double
    r2) {
    if (l1 > r1)
        swap(l1, r1);
    if (l2 > r2)
        swap(l2, r2);
    return max(l1, l2) <= min(r1, r2) + EPS;
}

int vec(const pt& a, const pt& b, const pt& c) {
    double s = (b.x - a.x) * (c.y - a.y) - (b.y - a.y) *
        (c.x - a.x);
    return abs(s) < EPS ? 0 : s > 0 ? +1 : -1;
}

bool intersect(const seg& a, const seg& b)
{
    return intersectld(a.p.x, a.q.x, b.p.x, b.q.x) &&
        intersectld(a.p.y, a.q.y, b.p.y, b.q.y) &&
        vec(a.p, a.q, b.p) * vec(a.p, a.q, b.q) <= 0 &&
        vec(b.p, b.q, a.p) * vec(b.p, b.q, a.q) <= 0;
}

bool operator<(const seg& a, const seg& b)
{
    double x = max(min(a.p.x, a.q.x), min(b.p.x, b.q.x));
    return a.get_y(x) < b.get_y(x) - EPS;
}

struct event {
    double x;
    int tp, id;

    event() {}
    event(double x, int tp, int id) : x(x), tp(tp), id(id)
        {}

    bool operator<(const event& e) const {
        if (abs(x - e.x) > EPS)
            return x < e.x;
        return tp > e.tp;
    }
};

set<seg> s;
vector<set<seg>::iterator> where;

set<seg>::iterator prev(set<seg>::iterator it) {
    return it == s.begin() ? s.end() : --it;
}

set<seg>::iterator next(set<seg>::iterator it) {
    return ++it;
}

pair<int, int> solve(const vector<seg>& a) {
    int n = (int)a.size();
    vector<event> e;
    for (int i = 0; i < n; ++i) {
        e.push_back(event(min(a[i].p.x, a[i].q.x), +1, i));
        e.push_back(event(max(a[i].p.x, a[i].q.x), -1, i));
    }
}

```

```

}
sort(e.begin(), e.end());

s.clear();
where.resize(a.size());
for (size_t i = 0; i < e.size(); ++i) {
    int id = e[i].id;
    if (e[i].tp == +1) {
        set<seg>::iterator nxt = s.lower_bound(a[id]), prv
            = prev(nxt);
        if (nxt != s.end() && intersect(*nxt, a[id]))
            return make_pair(nxt->id, id);
        if (prv != s.end() && intersect(*prv, a[id]))
            return make_pair(prv->id, id);
        where[id] = s.insert(nxt, a[id]);
    } else {
        set<seg>::iterator nxt = next(where[id]), prv =
            prev(where[id]);
        if (nxt != s.end() && prv != s.end() &&
            intersect(*nxt, *prv))
            return make_pair(prv->id, nxt->id);
        s.erase(where[id]);
    }
}

return make_pair(-1, -1);
}

```

4.3 Graph

4.3.1 2Sat

```

/**
 * 1 based index for variables
 * F = (a op b) and (c op d) and ..... (y op z)
 * a, b, c ... are the variables
 * sat::satisfy() returns true if there is some
 * assignment(True/False)
 * for all the variables that make F = True
 * * init() at the start of every case
 */

namespace sat{
    const int MAX = 200010; /// number of variables * 2
    bool vis[MAX];
    vector<int> ed[MAX], rev[MAX];
    int n, m, ptr, dfs_t[MAX], ord[MAX], par[MAX];

    inline int inv(int x){
        return ((x) <= n ? (x + n) : (x - n));
    }

    /// Call init once
    void init(int vars){
        n = vars, m = vars << 1;
        for (int i = 1; i <= m; i++){
            ed[i].clear();
            rev[i].clear();
        }
    }

    /// Adding implication, if a then b ( a --> b )
    inline void add(int a, int b){
        ed[a].push_back(b);
        rev[b].push_back(a);
    }

    /// (a or b) is true --> OR(a,b)
    /// (~a or b) is true --> OR(inv(a),b)
    /// (a or ~b) is true --> OR(a,inv(b))
    /// (~a or ~b) is true --> OR(inv(a),inv(b))
}

```

```

inline void OR(int a, int b){
    add(inv(a), b);
    add(inv(b), a);
}

/// same rule as or
inline void AND(int a, int b){
    add(a, b);
    add(b, a);
}

/// same rule as or
void XOR(int a, int b){
    add(inv(b), a);
    add(a, inv(b));
    add(inv(a), b);
    add(b, inv(a));
}

/// same rule as or
inline void XNOR(int a, int b){
    add(a, b);
    add(b, a);
    add(inv(a), inv(b));
    add(inv(b), inv(a));
}

/// (x <= n) means forcing variable x to be true
/// (x = n + y) means forcing variable y to be false
inline void force_true(int x){
    add(inv(x), x);
}

inline void topsort(int s){
    vis[s] = true;
    for(int x : rev[s]) if(!vis[x]) topsort(x);
    dfs_t[s] = ++ptr;
}

inline void dfs(int s, int p){
    par[s] = p;
    vis[s] = true;
    for(int x : ed[s]) if (!vis[x]) dfs(x, p);
}

void build(){
    CLR(vis);
    ptr = 0;
    for(int i=m;i>=1;i--) {
        if (!vis[i]) topsort(i);
        ord[dfs_t[i]] = i;
    }
    CLR(vis);
    for (int i = m; i >= 1; i--){
        int x = ord[i];
        if (!vis[x]) dfs(x, x);
    }
}

/// Returns true if the system is 2-satisfiable and
/// returns the solution (vars set to true) in vector
/// res
bool satisfy(vector<int>& res){
    build();
    CLR(vis);

    for (int i = 1; i <= m; i++){
        int x = ord[i];
        if (par[x] == par[inv(x)]) return false;
        if (!vis[par[x]]){
            vis[par[x]] = true;
        }
    }
}

```

```

        vis[par[inv(x)]] = false;
    }
}
res.clear();
for (int i = 1; i <= n; i++){
    if (vis[par[i]]) res.push_back(i);
}
return true;
}
}

```

4.3.2 Centroid_decomp

```

vector<int> ed[MAX]; /// adjacency list of the input
tree
bool isCentroid[MAX]; /// if the node is already a
centroid of some part
int sub[MAX], cpar[MAX], clevel[MAX];
int dis[20][MAX]; /// dis[i][j] = distance of node j from
the root of the i'th level of decomposition

```

```

void calcSubTree(int s,int p) {
    sub[s] = 1;
    for(int x : ed[s]) {
        if(x == p or isCentroid[x]) continue;
        calcSubTree(x,s);
        sub[s] += sub[x];
    }
}

int nn; /// number of nodes in the part

int getCentroid(int s,int p) {
    for(int x : ed[s]) {
        if(!isCentroid[x] && x!=p && sub[x]>(nn/2)) return
        getCentroid(x,s);
    }
    return s;
}

```

```

void setDis(int s, int from, int p, int lev) {
    dis[from][s] = lev;
    for(int x : ed[s]) {
        if(x == p or isCentroid[x]) continue;
        setDis(x, from, s, lev+1);
    }
}

```

///complexity --> O(nlog(n))

```

void decompose(int s,int p,int lev) {
    calcSubTree(s,p);
    nn = sub[s];
    int c = getCentroid(s,p);
    setDis(c,lev,p,0);

    isCentroid[c] = true;
    cpar[c] = p;
    clevel[c] = lev;

    for(int x : ed[c]) {
        if(!isCentroid[x]) decompose(x,c,lev+1);
    }
}

```

```
int ans[MAX];
```

```

inline void update(int v) {
    int u = v;
    while(u!=-1) {
        ans[u] = min(ans[u], dis[clevel[u]][v]);
        u = cpar[u];
    }
}

```

```

}
}

inline int query(int v) {
    int ret = INF;
    int u = v;
    while(u != -1) {
        ret = min(ret, dis[clevel[u]][v]+ans[u]);
        u = cpar[u];
    }
    return ret;
}

int main() {
    decompose(1,-1,0);
    for(int i=1; i<=n; i++) ans[i] = INF;
    update(v);
    query(v);
    return 0;
}

```

4.3.3 articulation_point

```

#include <bits/stdc++.h>

using namespace std;

const int N = 1e5 + 10;
vector<int> g[N];
int vis[N], low[N], cut[N], now = 0, n, m;

void dfs(int u, int p) {
    low[u] = vis[u] = ++now; int ch = 0;
    for(int v : g[u]){
        if(v ^ p) {
            if(vis[v]) low[u] = min(low[u], vis[v]);
            else {
                ch++; dfs(v, u);
                low[u] = min(low[u], low[v]);
                if(p + 1 && low[v] >= vis[u]) cut[u] = 1;
                if(low[v] > vis[u]) {
                    printf("Bridge %d -- %d\n", u, v);
                }
            }
        }
    }
    if(p == -1 && ch > 1) cut[u] = 1;
}

void ArticulationPointAndBridge() {
    now = 0;
    for(int i = 0; i < n; i++) {
        if(!vis[i]) dfs(i, -1);
    }
}

```

4.3.4 bcc

```

/// clear ed[] every test case
/// tot -> total number of components
/// bcc[i] contains the nodes of the i'th component
/// any self loop or multiple edge?

const int MAX = ?;
vector<int> ed[MAX];
bool cut[MAX];
int tot, Time, low[MAX], st[MAX];
vector<int> bcc[MAX];
stack<int> S;

void popBCC(int s,int x) {
    cut[s] = 1;
}

```

```

bcc[++tot].pb(s);
while(bcc[tot].back() ^ x) {
    bcc[tot].pb(S.top());
    S.pop();
}

void dfs(int s, int p = -1) {
    S.push(s);
    int ch = 0;
    st[s] = low[s] = ++Time;
    for(int x : ed[s]) {
        if(!st[x]) {
            ch++;
            dfs(x,s);
            low[s] = min(low[s],low[x]);
            if(p != -1 and low[x] >= st[s]) popBCC(s,x);
            else if(p == -1) if(ch > 1) popBCC(s,x);
        }
        else if(p != x) low[s] = min(low[s],st[x]);
    }
    if(p == -1 && ch > 1) cut[s] = 1;
}

```

```

void processBCC(int n) {
    for(int i=1;i<=n;i++) bcc[i].clear();
    CLR(st); CLR(cut);
    Time = tot = 0;
    for(int i=1; i<=n; i++) {
        if(!st[i]) {
            dfs(i,-1);
            if(!S.empty()) ++tot;
            while(!S.empty()) {
                bcc[tot].push_back(S.top());
                S.pop();
            }
        }
    }
}

```

4.3.5 bridge_tree

```

/**
 * 1 based indexing
 * call to processBridge(node,edges) generates bridge tree
 * and the edge list of that is brTree
 * Clear ed , isBridge , brTree per test case
 */

```

```

const int MAXN = ?;
const int MAXE = ?;

```

```

struct edges {
    int u,v;
} ara[MAXE];

```

```

vector<int> ed[MAXN]; /// actual graph
vector<int> isBridge[MAXN]; /// if the edge is a bridge,
the entry will be 1
vector<int> brTree[MAXN]; /// edges of the bridge tree

```

```

bool vis[MAXN];
int st[MAXN], low[MAXN], Time = 0;
int cnum; /// number of nodes in bridge tree
int comp[MAXN];

```

```

void findBridge(int s,int par) {
    int i,x,child = 0,j;
}

```

```

vis[s] = 1;
Time++;
st[s] = low[s] = Time;
for(i=0; i<ed[s].size(); i++) {
    x = ed[s][i];
    if(!vis[x]) {
        child++;
        findBridge(x,s);
        low[s] = min(low[s], low[x]);
        if(low[x] > st[s]) {
            isBridge[s][i] = 1;
            j = lower_bound(ed[x].begin(), ed[x].end(), s) - ed[x].begin();
            isBridge[x][j] = 1;
        }
    }
    else if(par!=x)
        low[s] = min(low[s], st[x]);
}
}

void dfs(int s) {
    int i,x;
    vis[s] = 1;
    comp[s] = cnum;
    for(i=0; i<ed[s].size(); i++) {
        if(!isBridge[s][i]) {
            x = ed[s][i];
            if(!vis[x]) dfs(x);
        }
    }
}

void processBridge(int n,int m) {
    CLR(vis);
    Time = 0;
    for(int i=1; i<=n; i++) if(!vis[i]) findBridge(i,-1);

    cnum = 0;
    CLR(vis);
    for(int i=1; i<=n; i++) {
        if(!vis[i]) {
            cnum++;
            dfs(i);
        }
    }

    n = cnum; ///number of nodes in the bridge tree

    for(int i=1; i<=m; i++) {
        if(comp[ara[i].u] != comp[ara[i].v]) {
            brTree[comp[ara[i].u]].pb(comp[ara[i].v]);
            brTree[comp[ara[i].v]].pb(comp[ara[i].u]);
        }
    }
}

int main() {
    int n,m,u,v;
    scanf("%d %d", &n, &m);
    for(int i=1; i<=m; i++) {
        sii(u,v);

        ed[u].pb(v);
        ed[v].pb(u);

        isBridge[u].pb(0);
        isBridge[v].pb(0);

        ara[i].u = u;
        ara[i].v = v;
    }
}

```

```

}
for(int i=1; i<=n; i++) sort(all(ed[i]));
processBridge(n,m);
return 0;
}

4.3.6 dsu_on_tree

vector<int> *vec[maxn];
int cnt[maxn];
void dfs(int v, int p, bool keep){
    int mx = -1, bigChild = -1;
    for(auto u : g[v])
        if(u != p && sz[u] > mx)
            mx = sz[u], bigChild = u;
    for(auto u : g[v])
        if(u != p && u != bigChild)
            dfs(u, v, 0);
    if(bigChild != -1)
        dfs(bigChild, v, 1), vec[v] = vec[bigChild];
    else
        vec[v] = new vector<int> ();
    vec[v]->push_back(v);
    cnt[ col[v] ]++;
    for(auto u : g[v])
        if(u != p && u != bigChild)
            for(auto x : *vec[u]){
                cnt[ col[x] ]++;
                vec[v] -> push_back(x);
            }
    ///now cnt[c] is the number of vertices in subtree of
    vertex v that has color c.
    /// note that in this step *vec[v] contains all of the
    subtree of vertex v.
    if(keep == 0)
        for(auto u : *vec[v])
            cnt[ col[u] ]--;
}

4.3.7 euler_path

/** 1-based *****/

vector<int> ed[MAX+5], sltn;
int inDeg[MAX+5], outDeg[MAX+5];
bool vis[MAX+5];

void dfs(int nd) {
    vis[nd] = true; /// used to check the connectivity of
    the graph
    while(ed[nd].size()) {
        int v = ed[nd].back();
        ed[nd].pop_back();
        dfs(v);
    }
    sltn.pb(nd);
}

/// returns 0 if no Euler path or circuit exists
/// returns 1 if a Euler trail exists
/// returns 2 if a Euler circuit exists
int findEuler(int n) {
    int src, snk, ret = 1;
    bool found_src = false, found_snk = false;
    CLR(inDeg); CLR(outDeg);
    for(int u = 1; u <= n; u++) {
        for(int i = 0; i<ed[u].size(); i++) {
            int v = ed[u][i];

```

```

            outDeg[u]++;
            inDeg[v]++;
        }
    }
    int diff;
    for(int i = 1; i<=n; i++) {
        diff = outDeg[i] - inDeg[i];
        if(diff == 1) {
            if(found_src) return 0;
            found_src = true;
            src = i;
        }
        else if (diff == -1) {
            if(found_snk) return 0;
            found_snk = true;
            snk = i;
        }
        else if(diff != 0) return 0;
    }
    if(!found_src) {
        /// there actually exists a euler cycle. So you need
        to pick a random node with non-zero degrees.
        ret = 2;
        for(int i = 1 ; i <= n ; i++) {
            if( outDeg[i] ) {
                found_src = true;
                src = i;
                break;
            }
        }
    }
    if(!found_src) return ret; /// every node has
    out-degree 0
    CLR(vis);
    sltn.clear();
    dfs(src);
    for(int i = 1; i<=n; i++) {
        /// the underlying graph is not even weakly
        connected.
        if(outDeg[i] && !vis[i]) return 0;
    }
    /// printing path
    for(int i = (int)sltn.size()-1; i>=0; i--) printf("%d\n", sltn[i]);
    puts("");
    return ret;
}

4.3.8 hld

int tt, tin[N], tout[N], sz[N], par[N][LG], hvc[N];
void dfs(int u, int p) {
    tin[u] = tt++;
    sz[u] = 1, par[u][0] = p;
    for(int j = 1; j < LG; ++j) {
        par[u][j] = par[par[u][j-1]][j-1];
    }
    int mx = 0;
    for(int &v : adj[u]) {
        if(v != p) {
            dfs(v, u);
            sz[u] += sz[v];
            if(sz[v] > mx) {
                mx = sz[v];
                hvc[u] = v;
            }
        }
    }
}

```

```

    tout[u] = tt - 1;
}
int ch_cnt, chno[N], chd[N], in[N], out[N];
void hld(int u, int p) {
    if (chd[ch_cnt] == -1) {
        chd[ch_cnt] = u;
    }
    chno[u] = ch_cnt, in[u] = tt++;
    if (hvc[u] != -1) {
        hld(hvc[u], u);
    }
    for (int &v : adj[u]) {
        if (v != p and v != hvc[u]) {
            ch_cnt++;
            hld(v, u);
        }
    }
    out[u] = tt - 1;
}

//pre_req
adj[u].clear();
hvc[u] = -1;
tt = 0;
dfs(0, 0);
chd[ch] = -1;
ch_cnt = 0, tt = 0;
hld(0, 0);

```

4.3.9 kuhn

```

/**
 * call init at the start of every test case
 * matchL[x] = y means node x of left side is matched to
 *   ↳ node y of right side
 * matchR[y] = x means node y of right side is matched to
 *   ↳ node x of left side
 *
 * y is in G[x] if there is an edge between node x and
 *   ↳ node y
 * Node x is in the left and node y is in the right side
 *
 * worst case complexity V*E
 */
namespace bpm {
    const int L = 105;
    const int R = 105;

    vector<int> G[L];
    int matchR[R], matchL[L], vis[L], it;

    /// n = number of nodes in the left side
    void init(int n) {
        SET(matchL), SET(matchR), CLR(vis);
        it = 1;
        for (int i = 1; i <= n; i++) G[i].clear();
    }

    inline void addEdge(int u, int v) { G[u].pb(v); }

    bool dfs(int s) {
        vis[s] = it;
        for (auto x : G[s]) {
            if (matchR[x] == -1 or (vis[matchR[x]] != it and
                ↳ dfs(matchR[x]))) {
                matchL[s] = x; matchR[x] = s;
                return true;
            }
        }
        return false;
    }
}

```

```

int solve() {
    int cnt = 0;
    for (int i = 1; i <= n; i++) {
        if (dfs(i)) cnt++, it++;
    }
    return cnt;
}

```

4.3.10 mcmf

**** I BASED NODE INDEXING. Comp : E * flow ****

```

namespace mcmf {
    using T = int;
    const T INF = ?; // 0x3f3f3f3f or 0x3f3f3f3f3f3f3fLL
    const int MAX = ?; // maximum number of nodes

    int n, src, snk;
    T dis[MAX], mCap[MAX];
    int par[MAX], pos[MAX];
    bool vis[MAX];

    struct Edge {
        int to, rev_pos;
        T cap, cost, flow;
    };

    vector<Edge> ed[MAX];

    void init(int n, int src, int snk) {
        n = n, src = src, snk = snk;
        for (int i = 1; i <= n; i++) ed[i].clear();
    }

    void addEdge(int u, int v, T cap, T cost) {
        Edge a = {v, (int)ed[v].size(), cap, cost, 0};
        Edge b = {u, (int)ed[u].size(), 0, -cost, 0};
        ed[u].pb(a);
        ed[v].pb(b);
    }

    inline bool SPFA() {
        CLR(vis);
        for (int i = 1; i <= n; i++) mCap[i] = dis[i] = INF;
        queue<int> q;
        dis[src] = 0;
        vis[src] = true; /// src is in the queue now
        q.push(src);

        while (!q.empty()) {
            int u = q.front();
            q.pop();
            vis[u] = false; /// u is not in the queue now
            for (int i = 0; i < (int)ed[u].size(); i++) {
                Edge &e = ed[u][i];
                int v = e.to;
                if (e.cap > e.flow && dis[v] > dis[u] + e.cost) {
                    dis[v] = dis[u] + e.cost;
                    par[v] = u;
                    pos[v] = i;
                    mCap[v] = min(mCap[u], e.cap - e.flow);
                    if (!vis[v]) {
                        vis[v] = true;
                        q.push(v);
                    }
                }
            }
        }
        return (dis[snk] != INF);
    }
}

```

```

}

inline pair<T, T> solve() {
    T F = 0, C = 0, f;
    int u, v;
    while (SPFA()) {
        u = snk;
        f = mCap[u];
        F += f;
        while (u != src) {
            v = par[u];
            ed[v][pos[u]].flow += f; // edge of v->u
            ↳ increases
            ed[u][ed[v][pos[u]].rev_pos].flow -= f;
            u = v;
        }
        C += dis[snk] * f;
    }
    return make_pair(F, C);
}

```

4.4 Math

4.4.1 FloorSum

```

// 0(log a) sum^n floor(ax + b / c) = f
long long FloorSumAP(long long a, long long b, long long
    ↳ c, long long n) {
    if (!a) return (b / c) * (n + 1);
    if (a >= c or b >= c) return ((n * (n + 1)) / 2) *
        ↳ (a / c) + (n + 1) * (b / c) + FloorSumAP(a % c, b
        ↳ % c, c, n);
    long long m = (a * n + b) / c;
    return m * n - FloorSumAP(c, c - b - 1, a, m - 1);
}

// 0(log a) sum^n x * floor(ax + b / c) = g, sum^n
    ↳ floor(ax + b / c)^2 = h
struct dat {
    long long f, g, h;
    dat(long long f = 0, long long g = 0, long long h = 0)
        ↳ : f(f), g(g), h(h) {}
};

long long mul(long long a, long long b) {
    return (a * b) % MOD;
}

dat query(long long a, long long b, long long c, long
    ↳ long n) {
    if (!a) return {mul(n + 1, b / c), mul(mul(mul(b / c,
        ↳ n), n + 1), inv2), mul(mul(n + 1, b / c), b / c)};
    long long f, g, h;
    dat nxt;
    if (a >= c or b >= c) {
        nxt = query(a % c, b % c, c, n);
        f = (nxt.f + mul(mul(mul(n, n + 1), inv2), a / c) +
            ↳ mul(n + 1, b / c)) % MOD;
        g = (nxt.g + mul(a / c, mul(mul(n, n + 1), mul(2 * n
            ↳ + 1, inv6))) + mul(mul(b / c, mul(n, n + 1)),
            ↳ inv2)) % MOD;
        h = (nxt.h + 2 * mul(b / c, nxt.f) + 2 * mul(a / c,
            ↳ nxt.g) + mul(mul(a / c, a / c), mul(mul(n, n +
            ↳ 1), mul(2 * n + 1, inv6))) + mul(mul(b / c, b /
            ↳ c), n + 1) + mul(mul(a / c, b / c), mul(n, n +
            ↳ 1))) % MOD;
        return {f, g, h};
    }
}

```



```
long long m = (a * n + b) / c;
nxt = query(c, c - b - 1, a, m - 1);
f = (mul(m, n) - nxt.f) % MOD;
g = mul(mul(m, mul(n, n + 1)) - nxt.h - nxt.f, inv2);
h = (mul(n, mul(m, m + 1)) - 2 * nxt.g - 2 * nxt.f -
    f) % MOD;
return {f, g, h};
}
```

4.4.2 NOD

```
N = input()
primes = array containing primes till 10^6
ans = 1
for all p in primes :
    if p*p*p > N:
        break
    count = 1
    while N divisible by p:
        N = N/p
        count = count + 1
    ans = ans * count
if N is prime:
    ans = ans * 2
else if N is square of a prime:
    ans = ans * 3
else if N != 1:
    ans = ans * 4
```

4.4.3 Pollard Rho

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
typedef long long ll;
```

```
typedef unsigned long long ull;
```

```
namespace Rho {
    ull mul(ull a, ull b, ull mod) {
        ll ret = a * b - mod * (ll) (1.L / mod * a * b);
        return ret + mod * (ret < 0) - mod * (ret >= (ll)
            mod);
    }

    ull bigMod(ull a, ull e, ull mod) {
        ull ret = 1;
        while (e) {
            if (e & 1) ret = mul(ret, a, mod);
            a = mul(a, a, mod), e >>= 1;
        }
        return ret;
    }

    bool isPrime(ull n) {
        if (n < 2 or n % 6 % 4 != 1) return (n | 1) == 3;
        ull a[] = {2, 325, 9375, 28178, 450775, 9780504,
            1795265022};
        ull s = builtin_ctzll(n - 1), d = n >> s;
        for (ull x : a) {
            ull p = bigMod(x % n, d, n), i = s;
            while (p != 1 and p != n - 1 and x % n and i--) p
                = mul(p, p, n);
            if (p != n - 1 and i != s) return 0;
        }
        return 1;
    }

    ull pollard(ull n) {
        auto f = [&](ull x) {return mul(x, x, n) + 1;};
}
```

```
ull x = 0, y = 0, t = 0, prod = 2, i = 1, q;
while (t++ % 40 or __gcd(prod, n) == 1) {
    if (x == y) x = ++i, y = f(x);
    if ((q = mul(prod, max(x, y) - min(x, y), n))
        prod = q;
    x = f(x), y = f(f(y));
}
return __gcd(prod, n);

vector<ull> factor(ull n) {
    if (n == 1) return {};
    if (isPrime(n)) return {n};
    ull x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), r.begin(), r.end());
    return l;
}

int t; ll n;

int main() {
    cin >> t;
    while (t--) {
        scanf("%lld", &n);
        vector<ull> facs = Rho::factor(n);
        sort(facs.begin(), facs.end());
        printf("%d", (int) facs.size());
        for (auto it : facs) printf(" %llu", it);
        puts("");
    }
    return 0;
}
```

4.4.4 catalan

```
//Recursive
const int MOD = ....
const int MAX = ....
int catalan[MAX];
void init() {
    catalan[0] = catalan[1] = 1;
    for (int i=2; i<=n; i++) {
        catalan[i] = 0;
        for (int j=0; j < i; j++) {
            catalan[i] += (catalan[j] * catalan[i-j-1])
                % MOD;
            if (catalan[i] >= MOD) {
                catalan[i] -= MOD;
            }
        }
    }
}
```

```
//Analytical formula:
```

```
ans = ncr(2*n,n)-ncr(2*n,n-1)= ncr(2*n,n)/(n+1)
```

4.4.5 crt

```
//r[i][j]= inverse of p[i] modulo p[j]
//ans= x[0]+x[1]*p[0]+x[2]*(p[0]*p[1])+...+x[k-1]*(p[0]*
    p[1]*p[2]*...*p[k-2])
//ans %= ((p[0]*p[1]*p[2]*...*p[k-1])

for (int i = 0; i < k; ++i) {
    x[i] = a[i];
    for (int j = 0; j < i; ++j) {
        x[i] = r[j][i] * (x[i] - x[j]);
    }
}
```

```
x[i] = x[i] % p[i];
if (x[i] < 0)
    x[i] += p[i];
}

ll mul= p[0],res=x[0],tot=1;
F(i,0,k){tot *= p[i];
F(i,1,k){
    res+= x[i]*mul;
    res %= tot;
    mul *= p[i];
}
res %= mul;
return res;
```

4.4.6 derangement

```
int derangement(int n)
{
    if(!n) return n;
    if(n <= 2) return n-1;
    return (n-1)*(derangement(n-1) + derangement(n-2));
}
```

4.4.7 diophantine

```
void print_solution(int a, int b, int c)
{
    int x, y;
    if (a == 0 && b == 0) {
        if (c == 0) {
            cout << "Infinite Solutions Exist" << endl;
        }
        else {
            cout << "No Solution exists" << endl;
        }
    }
    int gcd = gcd_extend(a, b, x, y);
    if (c % gcd != 0) {
        cout << "No Solution exists" << endl;
    }
    else {
        cout << "x = " << x * (c / gcd) << ", y = " << y
            * (c / gcd) << endl;
    }
}
```

4.4.8 discrete_log

```
// Returns minimum x for which a ^ x % m = b % m, a and m
    are coprime.
int solve(int a, int b, int m) {
    a %= m, b %= m;
    int n = sqrt(m) + 1;

    int an = 1;
    for (int i = 0; i < n; ++i)
        an = (an * 1ll * a) % m;

    unordered_map<int, int> vals;
    for (int q = 0, cur = b; q <= n; ++q) {
        vals[cur] = q;
        cur = (cur * 1ll * a) % m;
    }
}
```

```

    for (int p = 1, cur = 1; p <= n; ++p) {
        cur = (cur * 1ll * an) % m;
        if (vals.count(cur)) {
            int ans = n * p - vals[cur];
            return ans;
        }
    }
    return -1;
}

```

4.4.9 factorial_mod_p

```

// O(log p(n)) gives me n! % p for large n, p
int factmod(int n, int p) {
    vector<int> f(p);
    f[0] = 1;
    for (int i = 1; i < p; i++)
        f[i] = f[i-1] * i % p;

    int res = 1;
    while (n > 1) {
        if ((n/p) % 2)
            res = p - res;
        res = res * f[n%p] % p;
        n /= p;
    }
    return res;
}

```

4.4.10 fft

```

typedef complex<double> base;
#define PI acos(-1)
void fft(vector<base> &a, bool invert){
    int n = (int)a.size();
    for (int i = 1, j = 0; i < n; ++i){
        int bit = n >> 1;
        for (; j >= bit; bit >>= 1) j -= bit;
        j += bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <= 1){
        double ang = 2 * PI / len * (invert ? -1 : 1);
        base wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len){
            base w(1);
            for (int j = 0; j < len / 2; ++j){
                base u = a[i + j], v = a[i + j + len / 2] * w;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
                w *= wlen;
            }
        }
        if (invert) for (int i = 0; i < n; ++i) a[i] /= n;
    }
}

void multiply(const vector<int> &a, const vector<int> &b, vector<int> &res){
    vector<base> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    size_t n = 1;
    while (n < max(a.size(), b.size())) n <= 1;
    fa.resize(n), fb.resize(n);
    fft(fa, false), fft(fb, false);
    for (size_t i = 0; i < n; ++i) fa[i] *= fb[i];
    fft(fa, true); res.resize(n);
}

```

```

for (size_t i = 0; i < n; ++i) res[i] =
    int(fa[i].real() + 0.5);
}

```

4.4.11 gauss_elimination

```

const double EPS = 1e-9;
const int INF = 2; // it doesn't actually have to be
    infinity or a big number

int gauss (vector < vector<double> > a, vector<double> &
    ans) {
    int n = (int) a.size();
    int m = (int) a[0].size() - 1;

    vector<int> where (m, -1);
    for (int col=0, row=0; col<m && row<n; ++col) {
        int sel = row;
        for (int i=row; i<n; ++i)
            if (abs (a[i][col]) > abs (a[sel][col]))
                sel = i;
        if (abs (a[sel][col]) < EPS)
            continue;
        for (int i=col; i<=m; ++i)
            swap (a[sel][i], a[row][i]);
        where[col] = row;

        for (int i=0; i<n; ++i)
            if (i != row) {
                double c = a[i][col] / a[row][col];
                for (int j=col; j<=m; ++j)
                    a[i][j] -= a[row][j] * c;
            }
        ++row;
    }

    ans.assign (m, 0);
    for (int i=0; i<m; ++i)
        if (where[i] != -1)
            ans[i] = a[where[i]][m] / a[where[i]][i];

    for (int i=0; i<n; ++i) {
        double sum = 0;
        for (int j=0; j<m; ++j)
            sum += ans[j] * a[i][j];
        if (abs (sum - a[i][m]) > EPS)
            return 0;
    }

    for (int i=0; i<m; ++i)
        if (where[i] == -1)
            return INF;
    return 1;
}

//modular
int gauss (vector < bitset<N> > a, int n, int m,
    bitset<N> & ans) {
    vector<int> where (m, -1);
    for (int col=0, row=0; col<m && row<n; ++col) {
        for (int i=row; i<n; ++i)
            if (a[i][col]) {
                swap (a[i], a[row]);
                break;
            }
        if (! a[row][col])
            continue;
        where[col] = row;

        for (int i=0; i<n; ++i)
            if (i != row && a[i][col])

```

```

        a[i] ^= a[row];
    }
    ++row;
}

// The rest of implementation is the same as
    above

//rank
const double EPS = 1E-9;

int compute_rank(vector<vector<double>> A) {
    int n = A.size();
    int m = A[0].size();

    int rank = 0;
    vector<bool> row_selected(n, false);
    for (int i = 0; i < m; ++i) {
        int j;
        for (j = 0; j < n; ++j) {
            if (!row_selected[j] && abs(A[j][i]) > EPS)
                break;
        }

        if (j != n) {
            ++rank;
            row_selected[j] = true;
            for (int p = i + 1; p < m; ++p)
                A[j][p] /= A[j][i];
            for (int k = 0; k < n; ++k) {
                if (k != j && abs(A[k][i]) > EPS) {
                    for (int p = i + 1; p < m; ++p)
                        A[k][p] -= A[j][p] * A[k][i];
                }
            }
        }
    }
    return rank;
}

```

4.4.12 gen_all_k_combs

```

vector<int> ans;

void gen(int n, int k, int idx, bool rev) {
    if (k > n || k < 0)
        return;

    if (!n) {
        for (int i = 0; i < idx; ++i) {
            if (ans[i])
                cout << i + 1;
        }
        cout << "\n";
        return;
    }

    ans[idx] = rev;
    gen(n - 1, k - rev, idx + 1, false);
    ans[idx] = !rev;
    gen(n - 1, k - !rev, idx + 1, true);
}

void all_combinations(int n, int k) {
    ans.resize(n);
    gen(n, k, 0, false);
}

```

4.4.13 matrix_expo

```
struct Matrix{
    int sz;
    vector<vector<long long>> mat;
    Matrix(vector<vector<long long>> m){
        sz=m.size();
        mat=m;
    }

    Matrix operator *(Matrix &other){
        Matrix product(mat);
        for(int i=0;i<sz;i++){
            for(int j=0;j<sz;j++){
                product.mat[i][j]=0;
            }
            for(int i=0;i<sz;i++){
                for(int j=0;j<sz;j++){
                    for(int k=0;k<sz;k++){
                        product.mat[i][k]=(product.mat[i][k]+mat[i][j]*other.mat[j][k])%mod;
                    }
                }
            }
            return product;
        }
    };

    Matrix expo(Matrix &m,ll n){
        Matrix res(m.mat);

        for(int i=0;i<m.sz;i++){
            for(int j=0;j<m.sz;j++){
                if(i==j)res.mat[i][j]=1;
                else res.mat[i][j]=0;
            }
        }

        while(n){
            if(n&1)res=res*m;
            n>>= 1;
            m=m*m;
        }

        return res;
    }
};
```

4.4.14 next_lexicographical_k_comb

```
bool next_combination(vector<int>&a, int n) {
    int k = (int)a.size();
    for (int i = k - 1; i >= 0; i--) {
        if (a[i] < n - k + i + 1) {
            a[i]++;
            for (int j = i + 1; j < k; j++)
                a[j] = a[j - 1] + 1;
            return true;
        }
    }
    return false;
}
```

4.4.15 ntt

```
const int mod = 998244353;
const int root = 15311432;
const int k = 1 << 23;
int root_1;
vector<int> rev;
void pre(int sz){
    root_1 = bigmod(root, mod - 2, mod);
```

```
if (rev.size() == sz) return;
rev.resize(sz);
rev[0] = 0;
int lg_n = __builtin_ctz(sz);
for (int i = 1; i < sz; ++i)
    rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (lg_n - 1));
}

void fft(vector<int> &a, bool inv){
    int n = a.size();
    for (int i = 1; i < n - 1; ++i) if (i < rev[i])
        swap(a[i], a[rev[i]]);
    for (int len = 2; len <= n; len <= 1) {
        int wlen = inv ? root_1 : root;
        for (int i = len; i < k; i <= 1) wlen = 1ll *
            wlen * wlen % mod;
        for (int st = 0; st < n; st += len){
            int w = 1;
            for (int j = 0; j < len / 2; j++){
                int ev = a[st + j];
                int od = 1ll * a[st + j + len / 2] * w %
                    mod;
                a[st + j] = ev + od < mod ? ev + od : ev
                    + od - mod;
                a[st + j + len / 2] = ev - od >= 0 ? ev
                    - od : ev - od + mod;
                w = 1ll * w * wlen % mod;
            }
        }
    }

    if (inv){
        int n_1 = bigmod(n, mod - 2, mod);
        for (int x : a) x = 1ll * x * n_1 % mod;
    }
}

vector<int> mul(vector<int> &a, vector<int> &b){
    int n = a.size(), m = b.size(), sz = 1;
    while (sz < n + m - 1) sz <= 1;
    vector<int> x(sz), y(sz), z(sz);
    for (int i = 0; i < sz; ++i){
        x[i] = i < n ? a[i] : 0;
        y[i] = i < m ? b[i] : 0;
    }
    pre(sz);fft(x, 0);fft(y, 0);
    for (int i = 0; i < sz; ++i) z[i] = 1ll * x[i] *
        y[i] % mod;
    fft(z, 1);z.resize(n + m - 1);
    return z;
}
```

4.4.16 seg_sieve

```
/*
Segmented Sieve
This code was for 1 <= a <= b <= 2^31-1
Change variable types appropriately.
*/
bool notPrime[ ? ];
void segmented_sieve(int a, int b)
{
    int p, f;
    mem(notPrime, 0);
    for (int i = 0; i < tot_prime; i++)
    {
        p = prime[i];
        if (a % p == 0)
            f = a;
        else
```

```
f = (a - (a % p) + p);
f = max(p * p, f);
for (unsigned j = f; j <= b; j += p)
    notPrime[j - a] = true;
}
if (a == 1)
    notPrime[0] = 1;
}
```

4.4.17 stirling

```
/// Finds the number ways to put n balls into k
    indistinguishable boxes such
    that no box is empty.

int stirling2(int n, int k)
{
    if(n < k)
        return 0;
    if(k == 1)
        return 1;
    if(dp[n][k] == dp[n][k])
        return dp[n][k];
    return dp[n][k] = stirling2(n-1,k-1) +
        stirling2(n-1,k)*k;
}

/// Finds the number of ways to put n elements into k
    cycles where no cycle
    is empty

int stirling1(int n, int k)
{
    dp[n][k] = stirling1(n-1,k-1) + stirling(n-1,k)*n-1;
}
```

4.4.18 stirling_number_of_the_second_kind

```
// 1 / k! * sum (-1)^i nCr(k, i) * (k - i) ^ n
ll f(int n, int k) {
    ll res = 0;
    for (int i = 0; i < k; ++i) {
        if (i & 1) res = (res - nCr(k, i) * bp(k - i, n,
            mod) % mod + mod) % mod;
        else res = (res + nCr(k, i) * bp(k - i, n, mod)
            % mod) % mod;
    }
    if (res < 0) res += mod;
    return res * ifac[k] % mod;
}
```

4.4.19 totient

```
int phi(int n) {
    int result = n;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0)
                n /= i;
            result -= result / i;
        }
    }
    if (n > 1)
        result -= result / n;
    return result;
}
```

```
void phi_1 to n(int n) {
    vector<int> phi(n + 1);
    phi[0] = 0;
    phi[1] = 1;
    for (int i = 2; i <= n; i++)
        phi[i] = i;

    for (int i = 2; i <= n; i++) {
        if (phi[i] == i) {
            for (int j = i; j <= n; j += i)
                phi[j] -= phi[j] / i;
        }
    }
}
```

4.4.20 xor_basis

```
int basis[d]; // basis[i] keeps the mask of the vector
// whose f value is i
int sz;
void insertVector(int mask) {
    for (int i = 0; i < d; i++) {
        if ((mask & 1 << i) == 0) continue;
        if (!basis[i]) { // If there is no basis vector with
            // the i'th bit set, then insert this vector into the
            // basis
            basis[i] = mask;
            ++sz;
            return;
        }
        mask ^= basis[i]; // Otherwise subtract the basis
        // vector from this vector
    }
}
```

4.5 String

4.5.1 aho_corasick

```
/**
 * Given n patterns and a text T, for every pattern
 * you have to output the number of times that pattern
 * appears in the text.
 *
 * Don't forget to call build() after adding all the
 * patterns
 * to the Aho Corasick trie.
 *
 * ans[i] contains the number of occurrences of the i'th
 * pattern
 *
 * link[x] = y means there is a suffix link from node x
 * to node y
 *
 * out_link[x] = y means we can go from x to y using the
 * suffix links
 * suppose the path is as follows : x , a , b , c , ..., y
 * No pattern ends in node a, b, c, ... but some pattern
 * ends at node y.
 *
 * After a call to build(), the trie becomes a DAG(except
 * node 0)
 * next[x][c] = y means if we are currently at node x and
 * the character
 * c arrives, we will go to node y.
 *
 * Suppose sum of the length of the characters is N. Text
 * length is also
 * at most N. If all the patterns are unique, total number
 * of occurrences
```

```
of all the patterns will not be more than " N sqrt(N) ".
***/
```

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
const int N = ?; // Total number of characters in
// pattern
const int A = ?; // Alphabet size
```

```
struct AC {
    int nd, pt;

    int next[N][A], link[N], out_link[N], cnt[N], ans[N];
    vector<int> ed[N], out[N];

    AC(): nd(0), pt(0) { node(); }
```

```
int node() {
    memset(next[nd], 0, sizeof next[nd]);
    link[nd] = out_link[nd] = cnt[nd] = 0;
    ed[nd].clear(), out[nd].clear();
    return nd++;
}
```

```
void clear() {
    nd = pt = 0;
    node();
}
```

```
inline int get(char c) { return c - 'a'; }
```

```
void insert(const string &T) {
    int u = 0;
    for (char c : T) {
        if (!next[u][get(c)]) next[u][get(c)] = node();
        u = next[u][get(c)];
    }
    ans[pt] = 0;
    out[u].push_back(pt++);
}
```

```
void build() {
    queue<int> q;
    for (q.push(0); !q.empty(); ) {
        int u = q.front();
        q.pop();
        for (int c = 0; c < A; ++c) {
            int v = next[u][c];
            if (!v) next[u][c] = next[link[u]][c];
            else {
                link[v] = u ? next[link[u]][c] : 0;
                out_link[v] = out[link[v]].empty() ?
                // out_link[link[v]] : link[v];
                ed[link[v]].push_back(v);
                q.push(v);
            }
        }
    }
}
```

```
void dfs(int s) {
    for(int x : ed[s]) dfs(x), cnt[s] += cnt[x];
    for(int e : out[s]) ans[e] = cnt[s];
}
```

```
void traverse(const string &S) {
    int u = 0;
    for (char c : S) {
        u = next[u][get(c)];
        cnt[u]++;
    }
}
```

```
dfs(0);
}
};

char str[1000010], pat[505];

int main() {
    // freopen("in.txt", "r", stdin);
    AC aho;
    int t, T;
    scanf("%d", &T);
    for(int t=1; t<=T; t++) {
        int n;
        scanf("%d", &n);
        scanf("%s", str);
        for(int i=1; i<=n; i++) {
            scanf("%s", pat);
            aho.insert(pat);
        }
        aho.build();
        aho.traverse(str);
        printf("Case %d:\n", t);
        for(int i=0; i<n; i++) {
            printf("%d\n", aho.ans[i]);
        }
        aho.clear();
    }
    return 0;
}
```

4.5.2 hash

```
struct Hash {
    struct base {
        string s; int b, mod;
        vector<int> hash, p;
        void init(string &s, int _b, int _mod) { // b > 26,
            // prime.
            s = s; b = _b, mod = _mod;
            hash.resize(s.size());
            p.resize(s.size());
            hash[0] = s[0] - 'A' + 1; p[0] = 1;
            for(int i = 1; i < s.size(); ++i) {
                hash[i] = (ll) hash[i - 1] * b % mod;
                hash[i] += s[i] - 'A' + 1;
                if(hash[i] >= mod) hash[i] -= mod;
                p[i] = (ll) p[i - 1] * b % mod;
            }
        }
        int get(int l, int r) {
            int ret = hash[r];
            if(l) ret -= (ll) hash[l - 1] * p[r - l + 1] % mod;
            if(ret < 0) ret += mod;
            return ret;
        }
    } h[2];
    void init(string &s) {
        h[0].init(s, 29, 1e9+7);
        h[1].init(s, 31, 1e9+9);
    }
    pair<int, int> get(int l, int r) {
        return { h[0].get(l, r), h[1].get(l, r) };
    }
} H;
```

4.5.3 hash_segtree

```
/**
 * Everything is 0 based
```

```

* Call precal() once in the program
* Call update(1,0,n-1,i,j,val) to update the value of
  ↳ position
i to j to val, here n is the length of the string
* Call query(1,0,n-1,L,R) to get a node containing hash
of the position [L:R]
* Before any update/query
- Call init(str) where str is the string to be hashed
- Call build(1,0,n-1)
***/

```

```
#define INVALID_CHAR -1
```

```

namespace strhash {
    int n;
    const int MAX = 100010;
    int ara[MAX];
    const int MOD[] = {1067737007, 1069815139};
    const int BASE[] = {982451653, 984516781};

    int BP[2][MAX], CUM[2][MAX];

    void init(char *str) {
        n = strlen(str);
        for(int i=0; i<n; i++) ara[i] = str[i]-'0'+1; // scale
        ↳ str[i] if needed
    }

    void precal() {
        BP[0][0] = BP[1][0] = 1;
        CUM[0][0] = CUM[1][0] = 1;
        for(int i=1; i<MAX; i++) {
            BP[0][i] = ( BP[0][i-1] * (long long) BASE[0] ) %
                ↳ MOD[0];
            BP[1][i] = ( BP[1][i-1] * (long long) BASE[1] ) %
                ↳ MOD[1];

            CUM[0][i] = ( CUM[0][i-1] + (long long) BP[0][i] )
                ↳ % MOD[0];
            CUM[1][i] = ( CUM[1][i-1] + (long long) BP[1][i] )
                ↳ % MOD[1];
        }
    }

    struct node {
        int sz;
        int h[2];
        node() {}
    } tree[4*MAX];

    int lazy[4*MAX];

    inline void lazyUpdate(int n, int st, int ed) {
        if(lazy[n] != INVALID_CHAR) {
            tree[n].h[0] = (lazy[n] * (long long)
                ↳ CUM[0][ed-st]) % MOD[0];
            tree[n].h[1] = (lazy[n] * (long long)
                ↳ CUM[1][ed-st]) % MOD[1];

            if(st != ed) {
                lazy[2*n] = lazy[n];
                lazy[2*n+1] = lazy[n];
            }
            lazy[n] = INVALID_CHAR;
        }
    }

    inline node Merge(node a, node b) {
        node ret;

```

```

        ret.h[0] = ( ( a.h[0] * (long long) BP[0][b.sz] ) +
            ↳ b.h[0] ) % MOD[0];
        ret.h[1] = ( ( a.h[1] * (long long) BP[1][b.sz] ) +
            ↳ b.h[1] ) % MOD[1];

        ret.sz = a.sz + b.sz;

        return ret;
    }

    inline void build(int n, int st, int ed) {
        lazy[n] = INVALID_CHAR;
        if(st==ed) {
            tree[n].h[0] = tree[n].h[1] = ara[st];
            tree[n].sz = 1;
            return;
        }
        int mid = (st+ed)>>1;
        build(n+n, st, mid);
        build(n+n+1, mid+1, ed);

        tree[n] = Merge(tree[n+n], tree[n+n+1]);
    }

    inline void update(int n, int st, int ed, int i, int j, int
        ↳ v) {
        lazyUpdate(n, st, ed);
        if(st>j or ed<i) return;
        if(st>=i and ed<=j) {
            lazy[n] = v;
            lazyUpdate(n, st, ed);
            return;
        }
        int mid = (st+ed)>>1;
        update(n+n, st, mid, i, j, v);
        update(n+n+1, mid+1, ed, i, j, v);

        tree[n] = Merge(tree[n+n], tree[n+n+1]);
    }

    inline node query(int n, int st, int ed, int i, int j) {
        lazyUpdate(n, st, ed);
        if(st>=i and ed<=j) return tree[n];
        int mid = (st+ed)/2;
        if(mid<i) return query(n+n+1, mid+1, ed, i, j);
        else if(mid>=j) return query(n+n, st, mid, i, j);
        else return Merge(query(n+n, st, mid, i, j), query(n+n+1,
            ↳ mid+1, ed, i, j));
    }
}

```

4.5.4 manacher

```

vector<int> d1(n); // maximum odd length palindrome
↳ centered at i
// here d1[i]=the palindrome has
↳ d1[i]-1 right characters from i
// e.g. for aba, d1[1]=2;

for (int i = 0, l = 0, r = -1; i < n; i++) {
    int k = (i > r) ? 1 : min(d1[l + r - i], r - i);
    while (0 <= i - k && i + k < n && s[i - k] == s[i +
        ↳ k]) {
        k++;
    }
    d1[i] = k--;
    if (i + k > r) {
        l = i - k;
        r = i + k;
    }
}

```

```

}
vector<int> d2(n); // maximum even length palindrome
↳ centered at i
// here d2[i]=the palindrome has
↳ d2[i]-1 right characters from i
// e.g. for abba, d2[2]=2;

for (int i = 0, l = 0, r = -1; i < n; i++) {
    int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i +
        ↳ 1);
    while (0 <= i - k - 1 && i + k < n && s[i - k - 1] ==
        ↳ s[i + k]) {
        k++;
    }
    d2[i] = k--;
    if (i + k > r) {
        l = i - k - 1;
        r = i + k;
    }
}

```

4.5.5 palindromic_tree

```

const int A = 26;
const int N = 300010;

char s[N]; long long ans;
int last, ptr, nxt[N][A], link[N], len[N], occ[N];

void feed (int at) {
    while (s[at - len[last] - 1] != s[at]) last =
        ↳ link[last];
    int ch = s[at] - 'a', temp = link[last];
    while (s[at - len[temp] - 1] != s[at]) temp =
        ↳ link[temp];
    if (!nxt[last][ch]) {
        nxt[last][ch] = ++ptr, len[ptr] = len[last] + 2;
        link[ptr] = len[ptr] == 1 ? 2 : nxt[temp][ch];
    }
    last = nxt[last][ch], ++occ[last];
}

int main() {
    len[1] = -1, len[2] = 0, link[1] = link[2] = 1, last =
        ↳ ptr = 2;
    scanf("%s", s + 1);
    for (int i = 1, n = strlen(s + 1); i <= n; ++i)
        ↳ feed(i);
    for (int i = ptr; i > 2; --i) ans = max(ans, len[i] *
        ↳ 1LL * occ[i]), occ[link[i]] += occ[i];
    printf("%lld\n", ans);
    return 0;
}

```

4.5.6 persistant trie

```

/**
    Given an array of size n, each value in array can be
    ↳ expressed using
    20 bits.
    Query : L R K
    max(a_i ^ K) for L <= i <= R
    ***/

const int MAX = 200010; // maximum size of array
const int B = 19; // maximum number of bits in a value -
↳ 1
int root[MAX], ptr = 0;

```



```

struct node {
    int ara[2], sum;
    node() {}
} tree[ MAX * (B+1) ];

void insert(int prevnode, int &curRoot, int val) {
    curRoot = ++ptr;
    int curnode = curRoot;
    for(int i = B; i >= 0; i--) {
        bool bit = val & (1 << i);
        tree[curnode] = tree[prevnode];
        tree[curnode].ara[bit] = ++ptr;
        tree[curnode].sum += 1;
        prevnode = tree[prevnode].ara[bit];
        curnode = tree[curnode].ara[bit];
    }
    tree[curnode] = tree[prevnode];
    tree[curnode].sum += 1;
}

int find_xor_max(int prevnode, int curnode, int x) {
    int ans = 0;
    for(int i = B; i >= 0; i--) {
        bool bit = x & (1 << i);
        if (tree[tree[curnode].ara[bit ^ 1]].sum >
            tree[tree[prevnode].ara[bit ^ 1]].sum) {
            curnode = tree[curnode].ara[bit ^ 1];
            prevnode = tree[prevnode].ara[bit ^ 1];
            ans = ans | (1 << i);
        }
        else {
            curnode = tree[curnode].ara[bit];
            prevnode = tree[prevnode].ara[bit];
        }
    }
    return ans;
}

void solve() {
    int n, q, L, R, K;
    cin >> n;
    for(int i=1; i<=n; i++) cin >> ara[i];

    for(int i=1; i<=q; i++) {
        cin >> L >> R >> K;
        cout << find_xor_max(root[L-1], root[R], K) << endl;
    }
}

```

4.5.7 suffix_array

```

// Everything is 0-indexed
char s[N]; // Suffix array will be built for this string
int SA[N], iSA[N]; // SA is the suffix array, iSA[i]
    stores the rank of the i'th suffix
int cnt[N], nxt[N]; // Internal stuff
bool bh[N], b2h[N]; // Internal stuff
int lcp[N]; // Stores lcp of SA[i] and SA[i + 1]; lcp[n - 1] = 0
int lcpSparse[LOGN][N]; // lcpSparse[i][j] = min(lcp[j],
    ..., lcp[j - 1 + (1 << i)])

void buildSA(int n) {
    for (int i = 0; i < n; i++) SA[i] = i;
    sort(SA, SA + n, [](int i, int j) { return s[i] < s[j]; });

    for (int i = 0; i < n; i++) {
        bh[i] = i == 0 || s[SA[i]] != s[SA[i - 1]];
        b2h[i] = 0;
    }
}

```

```

}

for (int h = 1; h < n; h <= 1) {
    int tot = 0;
    for (int i = 0, j; i < n; i = j) {
        j = i + 1;
        while (j < n && !bh[j]) j++;
        nxt[i] = j; tot++;
    } if (tot == n) break;

    for (int i = 0; i < n; i = nxt[i]) {
        for (int j = i; j < nxt[i]; j++) iSA[SA[j]] = i;
        cnt[i] = 0;
    }

    cnt[iSA[n - h]]++;
    b2h[iSA[n - h]] = 1;
    for (int i = 0; i < n; i = nxt[i]) {
        for (int j = i; j < nxt[i]; j++) {
            int s = SA[j] - h;
            if (s < 0) continue;
            int head = iSA[s];
            iSA[s] = head + cnt[head]++;
            b2h[iSA[s]] = 1;
        }
        for (int j = i; j < nxt[i]; j++) {
            int s = SA[j] - h;
            if (s < 0 || !b2h[iSA[s]]) continue;
            for (int k = iSA[s] + 1; !bh[k] && b2h[k]; k++)
                b2h[k] = 0;
        }
        for (int i = 0; i < n; i++) {
            SA[iSA[i]] = i;
            bh[i] |= b2h[i];
        }
    }
    for (int i = 0; i < n; i++) iSA[SA[i]] = i;
}

void buildLCP(int n) {
    for (int i = 0, k = 0; i < n; i++) {
        if (iSA[i] == n - 1) {
            k = 0;
            lcp[n - 1] = 0;
            continue;
        }
        int j = SA[iSA[i] + 1];
        while (i + k < n && j + k < n && s[i + k] == s[j + k]) k++;
        lcp[iSA[i]] = k;
        if (k) k--;
    }
}

void buildLCPSparse(int n) {
    for (int i = 0; i < n; i++) lcpSparse[0][i] = lcp[i];
    for (int i = 1; i < LOGN; i++) {
        for (int j = 0; j < n; j++) {
            lcpSparse[i][j] = min(lcpSparse[i - 1][j],
                lcpSparse[i - 1][min(n - 1, j + (1 << (i - 1)))]);
        }
    }
}

pair<int, int> minLCPRange(int n, int from, int minLCP) {
    int r = from;
    for (int i = LOGN - 1; i >= 0; i--) {
        int jump = 1 << i;
    }
}

```

```

    if (r + jump < n && lcpSparse[i][r] >= minLCP) r += jump;
}

int l = from;
for (int i = LOGN - 1; i >= 0; i--) {
    int jump = 1 << i;
    if (l - jump >= 0 && lcpSparse[i][l - jump] >= minLCP) l -= jump;
}

return make_pair(l, r);
}

```

4.5.8 suffix_automata

```

/**
 * N = maximum possible string size
 * There won't be more than 2N - 1 nodes
 * There won't be more than 3N - 4 transitions
 * nodes are numbered from 0 to sz-1

 * scan sa::str
 * n = strlen(str)
 * call sa::build(n)

 * let's suppose sub_n represents the largest substring
    that is endpos equivalent to node n

 * cnt[i] = number of occurrences of sub_i in str
 * If terminal[i] = true, then sub_i is a suffix of str
 * There suffix link of node x to node y,
    Iff sub_y is the largest suffix of sub_x that is not
    endpos equivalent to node x.
 */

namespace sa {
    const int MAXN = 100005 << 1; // 2 * maximum possible
        string size
    const int MAXC = 26; // Size of the character set

    char str[MAXN];

    int n, sz, last; // sz = number of nodes in the
        automaton( node indexing is 0 based)
    int len[MAXN], link[MAXN], ed[MAXN][MAXC], cnt[MAXN];
    bool terminal[MAXN];
    vector<int> G[MAXN];

    void init() {
        SET(ed[0]);
        len[0] = 0, link[0] = -1, sz = 1, last = 0,
            terminal[0] = false;
    }

    inline int scale(char c) { return c - 'a'; }

    void extend(char c) {
        int cur = sz++;

        terminal[cur] = false;
        cnt[cur] = 1;

        SET(ed[cur]);
        len[cur] = len[last] + 1;
        int p = last;
        while (p != -1 && ed[p][c] == -1) {
            ed[p][c] = cur;
            p = link[p];
        }
        if (p == -1) link[cur] = 0;
        else {

```

```

int q = ed[p][c];
if (len[p] + 1 == len[q]) link[cur] = q;
else {
    int clone = sz++;
    len[clone] = len[p] + 1;
    memcpy(ed[clone], ed[q], sizeof(ed[q]));
    link[clone] = link[q];
    while (p != -1 && ed[p][c] == q) {
        ed[p][c] = clone;
        p = link[p];
    }
    link[q] = link[cur] = clone;

    cnt[clone] = 0;
    terminal[clone] = false;
}
}
last = cur;
}

// needed to generate cnt[]
void dfs(int s) {
    for(auto x : G[s]) dfs(x), cnt[s] += cnt[x];
}

void build() {
    init();
    int n = strlen(str);
    for(int i=0; i<n; i++) extend(scale(str[i]));

    // construction of cnt[]
    for(int i=1; i<sz; i++) G[link[i]].pb(i);
    dfs(0);
    for(int i=0; i<sz; i++) G[i].clear();

    // construction of terminal[]
    for(int i=last; i!=-1; i=link[i]) terminal[i] = true;
}
}

```

4.5.9 trie

```

#define N 200000 // total number of characters
// given as input
#define S 26

int root, now;
int nxt[N][S], cnt[N];

// will be called from main
void init(){
    root = now = 1;
    CLR(nxt), CLR(cnt);
}

inline int scale(char ch) { return (ch - 'a'); }

inline void Insert(char s[], int sz){
    int cur = root, to;
    for(int i=0; i<sz; i++){
        to = scale(s[i]);
        if( !nxt[cur][to] ) nxt[cur][to] = ++now;
        cur = nxt[cur][to];
    }
    cnt[cur]++;
}

inline bool Find(char s[], int sz){
    int cur = root, to;
    for(int i=0; i<sz; i++){
        to = scale(s[i]);
        if( !nxt[cur][to] ) return false;
        cur = nxt[cur][to];
    }
    return (cnt[cur]!=0);
}

// It's better to call the Delete() after checking if
// the

```

```

// string we wanna delete actually exists in the trie
inline void Delete(char s[], int sz){
    int cur = root, to;
    for(int i=0; i<sz; i++){
        to = scale(s[i]);
        cur = nxt[cur][to];
    }
    cnt[cur]--;
}

```

4.5.10 z_algo

```

const int N = 100010;

char s[N];
int t, n, z[N];

int main() {
    scanf("%s", s);
    n = strlen(s), z[0] = n;
    int L = 0, R = 0;
    for (int i = 1; i < n; ++i) {
        if (i > R) {
            L = R = i;
            while (R < n && s[R - L] == s[R]) ++R;
            z[i] = R - L; --R;
        } else {
            int k = i - L;
            if (z[k] < R - i + 1) z[i] = z[k];
            else {
                L = i;
                while (R < n && s[R - L] == s[R]) ++R;
                z[i] = R - L; --R;
            }
        }
    }
}

```